in28minutes.com · Interview Preparation Series

CLOUD

Interview Guide

8 Chapters · 61 Questions & Answers · June 2026

Table of Contents

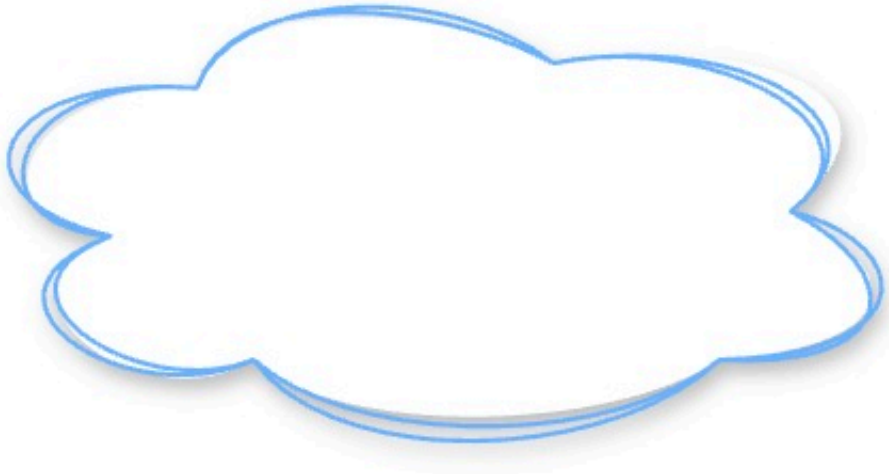
CHAPTER 1

Cloud Fundamentals

8 Questions & Answers

1 Cloud Fundamentals

Q1 What is Cloud? What are its advantages?



Cloud

What is Cloud?

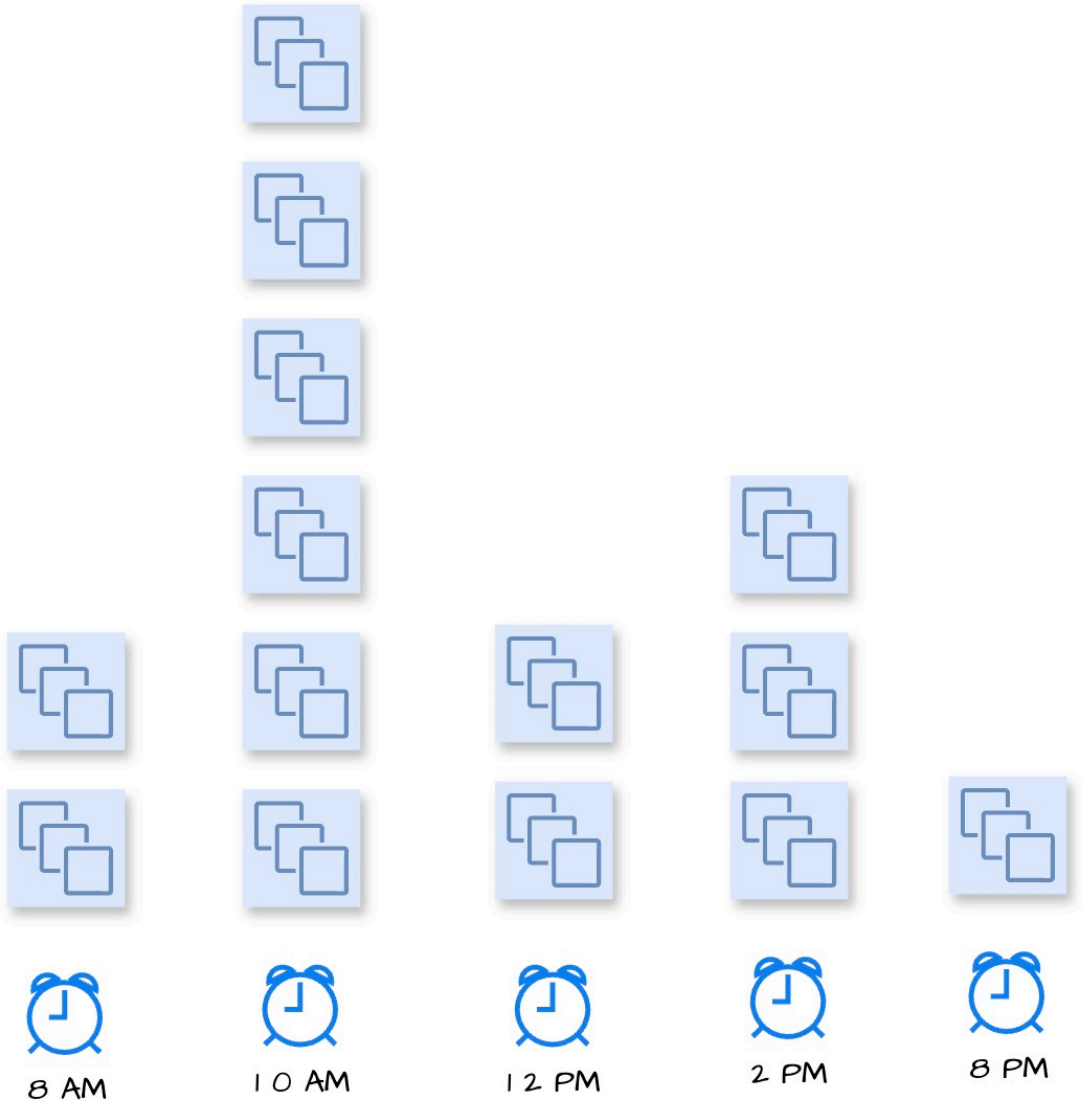
- Scenario: Imagine you need to setup 100 servers for a website you are building. Can we rent instead of buy?
- Cloud: Rent computing, storage, database, and other services from companies like AWS, Azure, Google Cloud, .. – whenever you need them.

Cloud is On-Demand

- No Upfront Investment: No need to buy servers or data centers
- Faster Start: Launch apps in minutes, not months
- Pay-As-You-Go: Pay only for what you use

Elasticity

Cloud is Scalable



- Elastic - Scale Up/Down: Add or remove resources as needed
- Automatic Scaling: Many services scale based on demand
- Global Reach: Deploy apps closer to your users across the world

Regions

Benefits of Cloud

- Faster Delivery: Launch apps in minutes
- Cost-Efficient: Pay only for what you use
- Scalable: Auto-scale to handle traffic spikes
- Global: Deploy close to users, anywhere in the world
- Secure & Compliant: Built-in security and certifications

Q2 Region vs Zones vs EdgeLocations

Region vs Zones vs Edge Locations

- Scenario: You want your app to be fast, reliable, and globally available. How does the cloud provider structure its infrastructure to help you do that?
- Answer: Using Regions, Zones, and Edge Locations – each plays a different role.

Region

- What It Is: A geographical area (e.g., Mumbai, US-East)
- Use Case: Choose a region close to your users or data for low latency and compliance

Regions

Region Advantages

- High Availability: Even if a region is down, you can serve from other regions
- Low Latency: Deploy closer to users for faster response
- Global Footprint: Reach users worldwide without owning infrastructure
- Adhere to government regulations: Keep data in-region to meet compliance laws

Zones

Zone

- What It Is: One or more data centers inside a region
- Isolated but Connected: Physically separated but connected with low latency
- Use Case: Deploy across zones for high availability and fault tolerance (while being in same region)

Edge Location

- What It Is: A content delivery endpoint near users
- Key Thing To Remember: Count of Edge Locations is far greater than Count of Regions
- Smaller than AZs: Focused on caching and request routing
- Use Case: Deliver content faster via CDN (e.g., CloudFront)

Regions

Edge Location Use Case – Step by Step

- Step 01: A user requests a video or image from your website (e.g., https://yourapp.com/logo.png)
- Step 02: The request is routed to the nearest edge location (a local server near the user)
- Step 03: If the content is cached at the edge, it is served instantly — no need to reach the main server
- Step 04: If not cached, the edge location fetches it from the origin server
- Step 05: The content is delivered to the user and simultaneously cached at the edge for future requests
- Step 06: Next time another user in the same region makes the same request, it's delivered directly from the edge — faster and more efficient
- Edge locations improve performance, reduce latency, and offload origin servers, especially for static content and media files

What are Multi Regions?

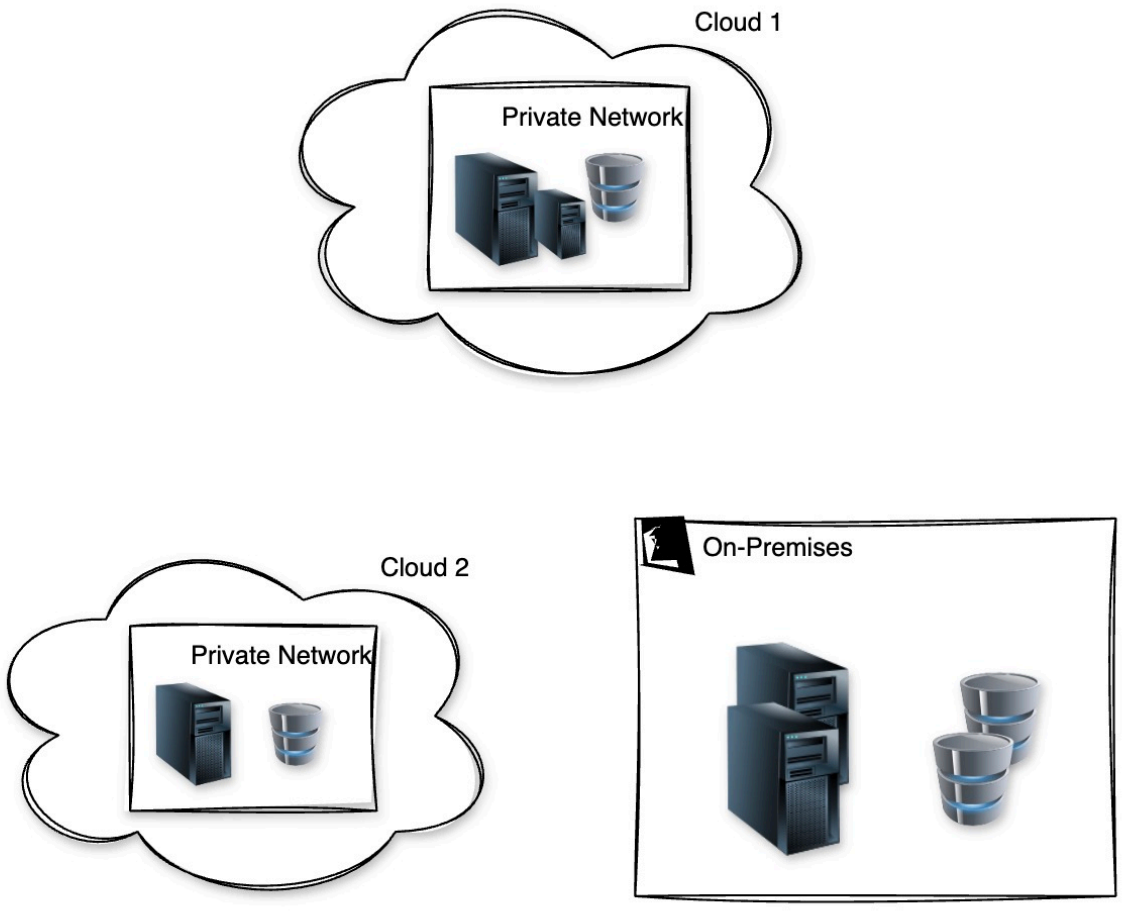
- Multi Regions: Multiple geographically separate cloud regions are logically linked to enable geo-redundancy, disaster recovery, and resilient architecture

Key Benefits of Multi Regions

- High Availability: Data is replicated to multiple regions
- High Durability: Multiple copies of data are stored across both regions

Concept	AWS	Azure
Region	Region E.g., us-east-1	Region E.g., East US, West Europe
Zone	Availability Zone E.g., us-east-1a	Availability Zone E.g., East US 2 Zone 1
Edge Location	Edge Location (CloudFront POP) E.g., Hyderabad	Point of Presence (POP) used in Azure Front Door / Azure CDN
Multi-Region / Dual-Region	Not explicitly named; use features like Cross-Region Replication, Global DynamoDB	Region Pairs (Dual Regions) Automatically paired for DR, etc.

Q3 On-premise vs Hybrid Cloud vs Multi Cloud



1. On-Premise

- What It Is: All infrastructure is owned and operated by you
- Where It Runs: In your own data center
- Pros:
  - Full control over hardware and security
  - No dependency on external providers
- Cons:
  - Expensive upfront cost
  - Hard to scale quickly
  - Slower to adopt modern tools

2. Hybrid Cloud

- What It Is: A mix of on-prem + public cloud
- Where It Runs: Some workloads on-prem, some in cloud
- Pros:
  - Flexibility to keep sensitive data on-prem
  - Leverage cloud scale for web apps or backups
  - Step-by-step cloud adoption
- Cons:
  - Complex setup and integration
  - Needs strong network and security architecture
  - Harder to manage consistently

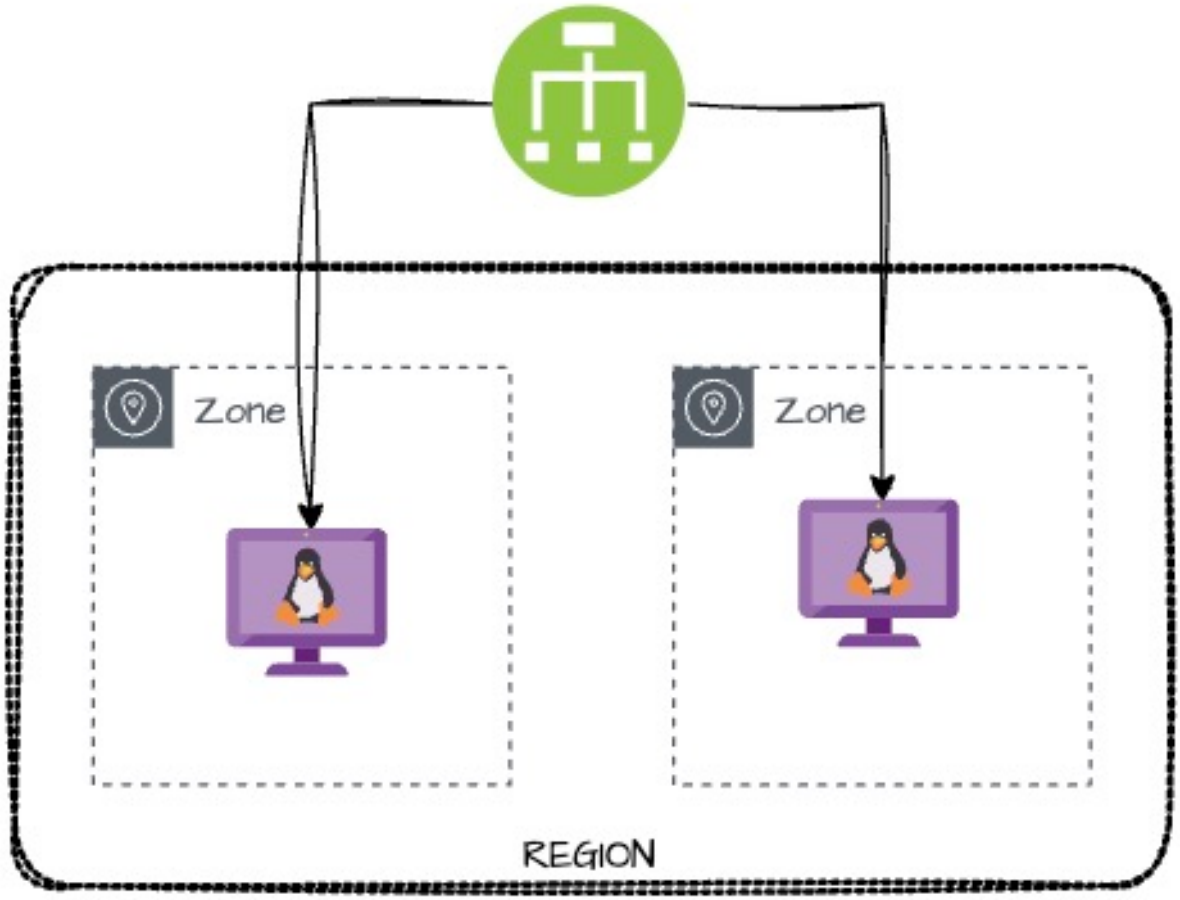
3. Multi-Cloud

- What It Is: Use of 2 or more public cloud providers (e.g., AWS + Azure) with or without on-premises
- Why Use It: Avoid vendor lock-in, optimize cost/performance
- Pros:
  - Use best features from each provider
  - Increase availability and redundancy
  - Competitive pricing and negotiation
- Cons:
  - Higher complexity in architecture and management
  - Requires multi-skilled teams
  - Data sync challenges

Q4 IaaS vs PaaS vs SaaS vs Serverless

IaaS vs PaaS vs SaaS vs Serverless

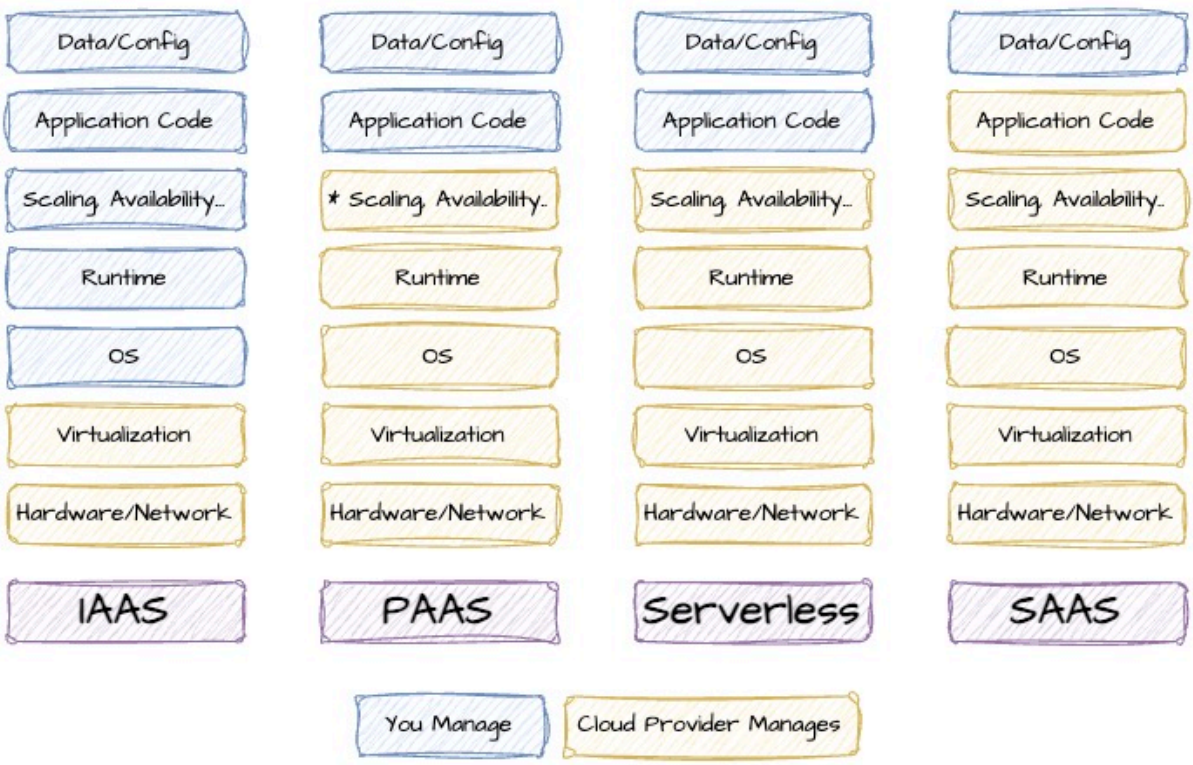
- Scenario: You want to build, deploy, or use an application. How much do you want to manage yourself vs how much should the cloud provider handle for you?
- Goal: Choose the right cloud service model based on control, convenience, and responsibility.



Load Balancing

IaaS (Infrastructure as a Service)

- Use only infrastructure from cloud provider
  Ex: Using VM service to deploy your apps/databases
- Cloud provider is responsible for:
  Hardware, Networking & Virtualization
- You are responsible for:
  OS upgrades and patches
  Application Code and Runtime
  Configuring load balancing
  Auto scaling
  Availability
  etc.. (and a lot of things!)
- Examples:Amazon EC2, Azure Virtual Machines, Google Compute Engine



Cloud Service Models

PaaS (Platform as a Service)

- Use a platform provided by the cloud
  - Cloud provider is responsible for:
  - Hardware, Networking & Virtualization
  - OS (incl. upgrades and patches)
  - Application Runtime
  - Auto scaling, Availability & Load balancing etc..
  - You are responsible for:
  - Configuration (of Application and Services)
  - Application code (if needed)
- Examples:
  - Compute: AWS Elastic Beanstalk, Azure App Service, Google App Engine
  - Databases: Relational (Amazon RDS, Google Cloud SQL, Azure SQL Database etc)
  - And a lot of others!

SaaS (Software as a Service)

- Centrally hosted software (mostly on the cloud)
  - Offered on a subscription basis (pay-as-you-go)
  - Examples:
  - Email, calendaring & office tools (such as Outlook 365, Microsoft Office 365, Gmail, Google Docs)
  - Customer relationship management (CRM), enterprise resource planning (ERP) and document management tools
- Cloud/Service provider is responsible for:
  - OS (incl. upgrades and patches)
  - Application Runtime
  - Auto scaling, Availability & Load balancing etc..
  - Application code and/or
  - Application Configuration (How much memory? How many instances? ..)
- Customer is responsible for:
  - Configuring the software!

Serverless

- What do we think about when we develop an application?
  - Where to deploy? What kind of server? What OS?
  - How do we take care of scaling and availability of the application?
- What if you don't worry about servers and focus ONLY on code?
  - Enter Serverless
  - Remember: Serverless does NOT mean "No Servers"
- Serverless for me:
  - You don't worry about infrastructure (ZERO visibility into infrastructure)
  - Flexible scaling and automated high availability
  - Most Important: Pay for use
  - Ideally ZERO USAGE => ZERO COST
- You focus on code and the cloud managed service takes care of all that is needed to scale your service/code to serve millions of requests!
  - And you pay for usage and NOT servers!

Serverless Examples

Category	AWS	Azure
Compute	AWS Lambda, AWS Fargate	Azure Functions, Azure Container Apps
Storage	Amazon S3	Azure Blob Storage
Databases and a lot of others	..	..

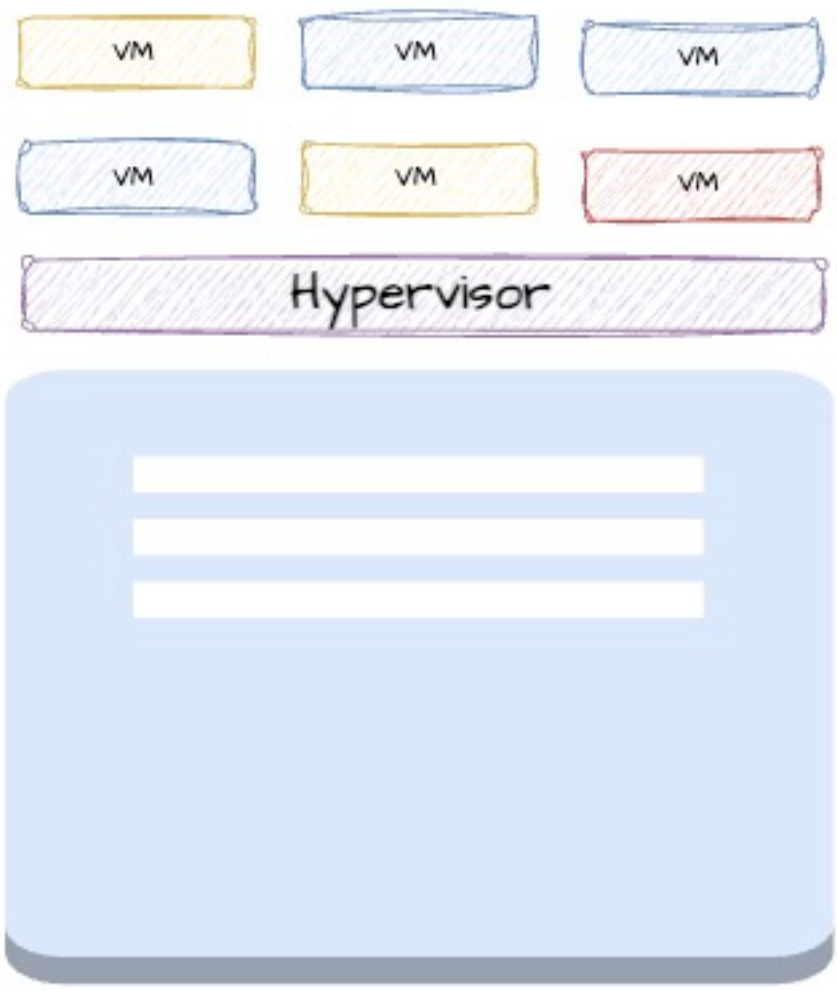
Q5 What is Virtualization?

What is Virtualization?

- Scenario: In the past, running one app meant using one full physical server—even if the app used only 10% of the CPU. This wasted resources and increased costs.
- Goal: Use one physical server to run multiple virtual systems efficiently.

Virtualization

Virtualization – The Basics



- Definition: Creating virtual versions of computing resources (like virtual machines) on a single physical server
- How It Works:
  - A Hypervisor (software layer) sits on top of the physical server
  - It splits the hardware into multiple Virtual Machines (VMs)
  - Each VM acts like a separate computer with its own OS and applications

Why Use Virtualization?

- Better Utilization: Run many apps on fewer servers
- Cost Efficiency: Lower hardware and maintenance costs
- Flexibility: Quickly create, start, stop, or move VMs
- Isolation: Each VM is independent – safer and easier to manage

Foundation of Cloud Computing

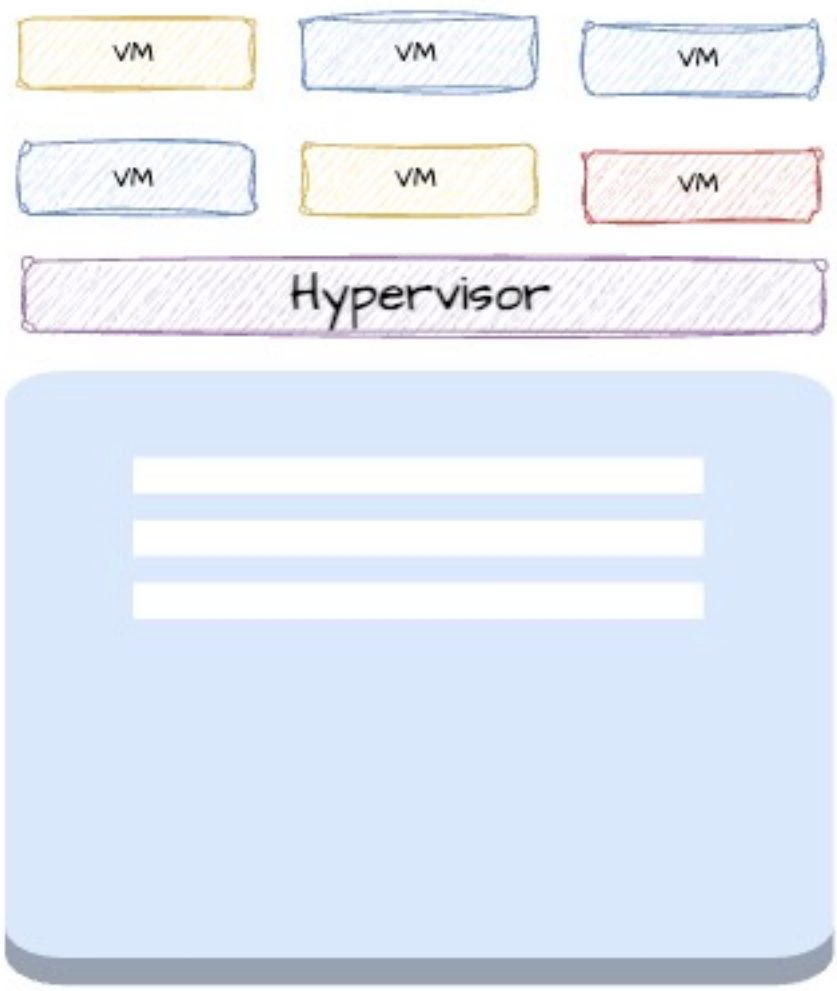
- Public cloud providers (like AWS, Azure, Google Cloud) use virtualization to offer shared infrastructure
- You get a VM (virtual server), not a dedicated physical machine
- Containers, serverless functions and almost everything you see in the cloud are built on top of virtualization layers

---

Q6 How can you create Virtual Machines in the cloud?

What is a Virtual Machine (VM)?

- Definition: A software-based (virtual) machine running inside a physical server
- Goal: Run multiple isolated systems on a single physical machine
- Benefit: Full control over OS, software, and security settings while sharing hardware



Virtualization

Cloud VMs are Flexible

- Choose What You Need: Select CPU, RAM, storage, and OS
- Multiple OS Options: Run Linux, Windows, or custom images
- Custom Machine Types: Match VMs to your workload and budget

Cloud VMs are Scalable

- Scale Up/Down: Change number of VMs based on traffic
- Auto Scaling: Add or remove VMs automatically
- Global Deployment: Launch in regions close to users

Cloud VMs are Cost Efficient

- Pay-As-You-Go: Billed by second, minute, or hour
- Reserved/Spot Pricing: Save cost with long-term reservations or using spare capacity
- Release When Not Needed: Reduce waste by releasing unused VMs

Reviewing Important Virtual Machine Concepts

Feature	Explanation	AWS	Azure
Managed Service	Create VMs	Amazon EC2 (Elastic Compute Cloud)	Azure Virtual Machines
Image	What OS and software should be on the instance?	AMI (Amazon Machine Image)	VM Image
Instance Family	Type of hardware: General, High CPU or High Memory or Storage Optimized	Instance Family	VM Series
Instance Size	Amount of vCPU and memory	Instance Type - t3.micro, m5.large	VM Sizes - B2s,B2ms ..
Attached Disks	Block storage volumes for VMs	Elastic Block Store	Managed Disks

Networking for VMs

Feature	Explanation	AWS	Azure
Permanent Internal IP Address	Permanent Internal IP Address that does not change during the lifetime of an instance	Private IP Address	Private IP Address
Ephemeral External IP Address	Ephemeral External IP Address that changes when an instance is stopped	Public IP Address	Public IP Address
Permanent External IP Address	Permanent External IP Address that can be attached to a VM	Elastic IP Address	Static IP Address
Firewall Rules	Control incoming/outgoing traffic	Security Group	Network Security Group (NSG)

> Managing Costs

Feature	Explanation	AWS	Azure
Cheaper temp instances	Create cheaper, temporary instances for non critical workloads	Spot instances	Spot VMs
Reservations/Usage Discounts	Reserve compute instances ahead of time/Get discounts for using resources for long periods of time	Reserved instances	Reserved instances
Reservation based on dollar amounts	Reserve based on monetary commitment ($100 per hour, for example)	AWS Savings Plans	Azure Savings Plan for Compute
Managing Budget	Budget Management	Budget alerts	Budget alerts

Q7 What are some of the architectural aspects to think about when creating virtual machines in the cloud?

- Scaling
- Resiliency
- Observability
- Tenancy

What is Scaling?

- Scenario: Your app is growing – more users, more traffic, more data. One server is not enough anymore. You need to scale.
- Scaling: Expanding system capacity to handle more load.

Scaling Types

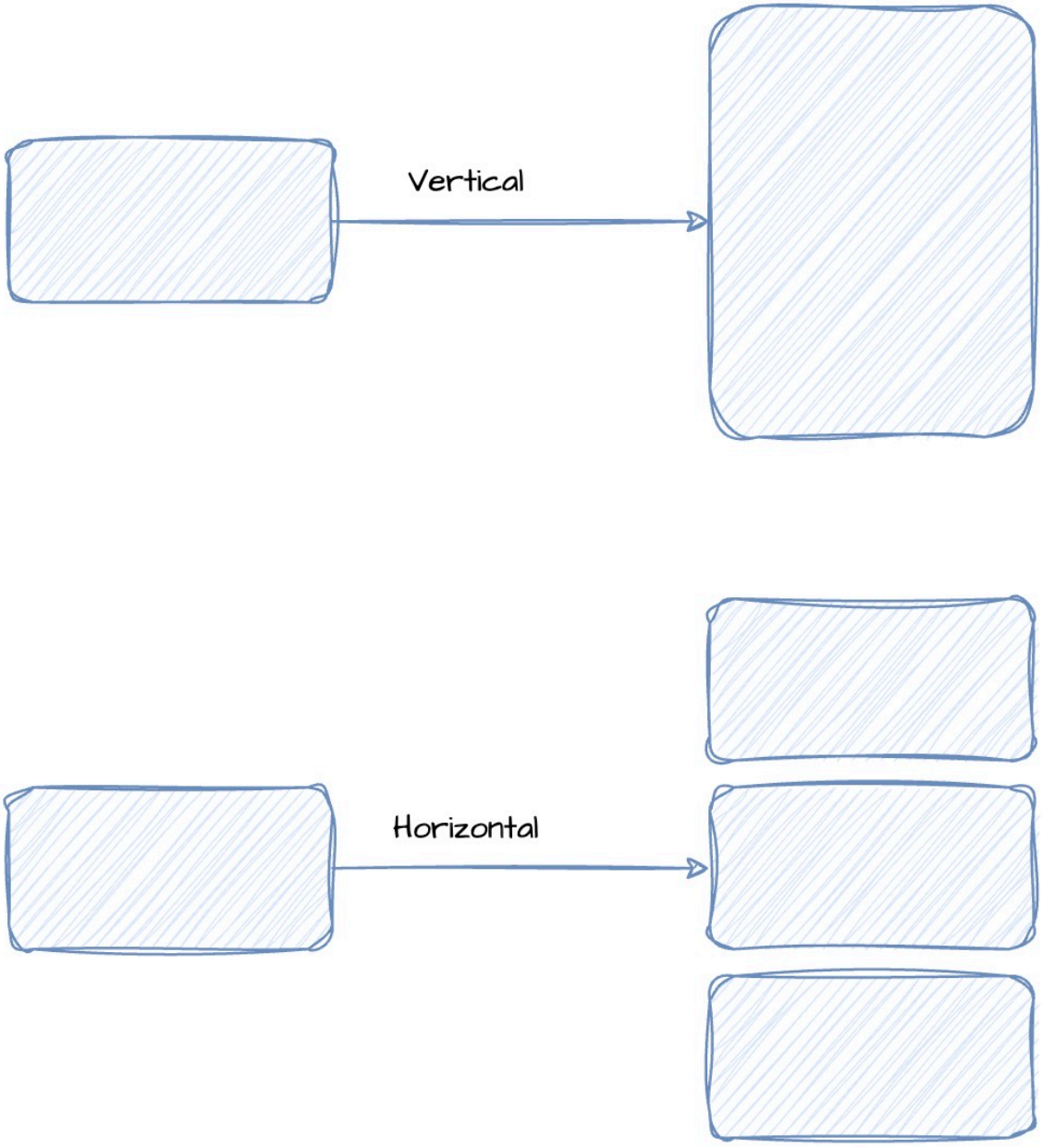
Vertical Scaling (Scale Up)

- What It Is: Increase the power of a single machine
  - More CPU, RAM, disk, or network
  - Example: Upgrade a small server to a large one
- Benefits:
  - Simple and fast to implement
  - No code changes or architectural updates
- Limitations:
  - Hardware has physical limits
  - Downtime may be required
  - Higher cost for high-end machines

Horizontal Scaling (Scale Out)

- What It Is: Add more machines or instances to share the load
  - Run multiple copies of the application or database
- Benefits:
  - Improved availability and fault tolerance
  - Scale without downtime
- Challenges:
  - Needs infrastructure like load balancers

Horizontal Scaling in Practice



- Auto Scaling: Automatically add or remove instances based on traffic
- Load Balancing: Evenly distribute requests across all instances
- Deployment Models:
  - Within a single zone
  - Across multiple zones

Resiliency for Virtual Machines and Load Balancing (Cloud Neutral)

- Scenario: You're running your application on virtual machines. What happens if a VM crashes, a zone goes down, or your app faces a sudden surge in traffic?
- Goal: Design your architecture to handle failures gracefully and continue to serve users reliably — that’s resiliency.

What is Resiliency?

- Definition: The ability of a system to provide acceptable performance and recover quickly even when parts of the system fail
- Why It Matters:
  - Failures are inevitable (hardware, software, network)
  - Users expect high uptime and reliability

Building Resilient Architectures

Design Element
Auto-Healing Groups / Instance Groups
Load Balancing
Multi-Zone Deployment
Health Checks
Disaster Recovery Planning

Creating Multiple VMs

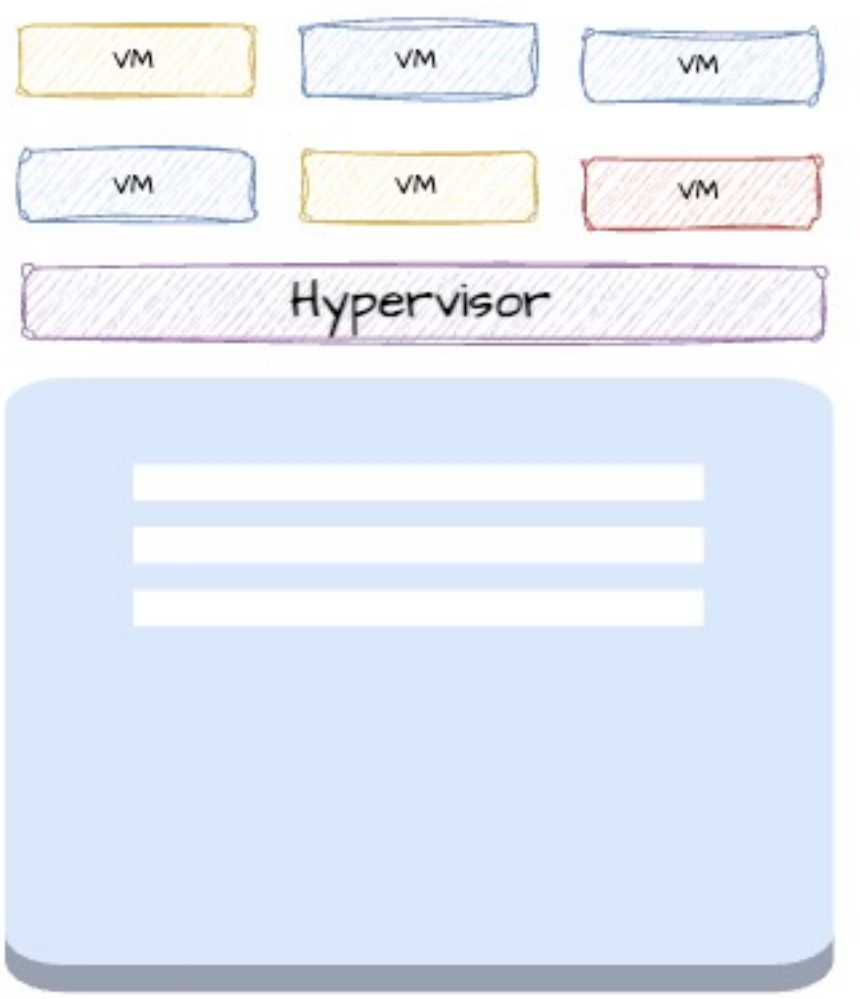
Feature	Explanation	AWS	Azure
VM Templates	Templates to simplify creation of Virtual Machines	Launch Templates	- (ARM Templates)
Auto Scaling	Automatically add or remove VMs based on usage (load, time, events)	Auto Scaling Group (ASG)	Virtual Machine Scale Sets (VMSS)
Load Balancing	Load Balancing	Elastic Load Balancer	Azure Load Balancer (& others)
VM Management	Simplify management (software, OS patches etc) of 1000's of Virtual Machines	Systems Manager	Azure Update Manager

> Observability
>
> - Collect metrics like CPU, memory, network usage, disk I/O
> - Capture application and system logs for troubleshooting
> - Set alerts for failures

Question	AWS	Azure
How do you monitor metrics around your applications	Amazon CloudWatch	Azure Monitor
How do you manage application and service logs?	Amazon CloudWatch Logs	Azure Monitor Logs
How do you trace requests across applications and services?	AWS X-Ray	Azure Application Insights Distributed Tracing

> Why Tenancy Matters?

- Scenario: You’re deploying an app with licensing restrictions or regulatory needs. You need control over the hardware it runs on.
- Tenancy: Determines who shares the physical server where your virtual machines (VMs) run.



Virtualization

Shared Tenancy (Default)

- What It Is: Multiple customers’ instances run on the same physical server
- Limitations:
  Not suitable for strict compliance or licensing needs

Dedicated Hosts

- What It Is: Entire physical server reserved for your use
- Benefits:
  - Required for some licensing models (e.g., Windows Server, SQL Server)
  - Helps meet compliance and audit requirements
- Limitations:
  - More expensive

Feature	Explanation	AWS	Azure
Host dedicated to one customer	Physical hosts dedicated to one customer	EC2 Dedicated Hosts	Azure Dedicated Hosts

---

Q8 What are the Compute Options in Cloud?

Category	AWS	Azure
IaaS	Amazon EC2	Azure VMs
PaaS	AWS Elastic Beanstalk	Azure App Service
FaaS - Serverless	AWS Lambda	Azure Functions
CaaS - Serverless	AWS Fargate	Azure Container Instances
CaaS - Kubernetes	Amazon EKS	Azure Kubernetes Service, Azure Red Hat OpenShift
CaaS - Custom	Amazon ECS	
Multi-cloud Container services	Amazon EKS Anywhere	Azure Arc-enabled Kubernetes
VMWare	VMware Cloud on AWS	Azure VMware Solution

CHAPTER 2

Cloud Storage and Databases Fundamentals

12 Questions & Answers

2 Cloud Storage and Databases Fundamentals

Q1 Compare Availability vs Durability vs Consistency

database horizontal scaling

Availability

- Definition: Will your application or data be accessible when you need it?
- Measured As: Uptime percentage over time (e.g., per month or year)
- Typical Targets:
  - 99.95% ~22 minutes of downtime/month
  - 99.99% (4 9's) ~4.5 minutes/month
  - 99.999% (5 9's) ~26 seconds/month
- How to Improve:
  - Have multiple copies/instances
  - Use multi-zone or multi-region deployments
  - Add load balancers and health checks
  - Implement replication and auto-failover

Durability

- Definition: Will your data still exist years from now, even through hardware failures or disasters?
- Measured As: Probability of data loss over time
- Typical Targets:
  99.999999999% (11 9's) durability
  Means: Store 1 million files for 10 million years, lose only 1 file
- Why It Matters:
  Once data is lost, it cannot be recovered
  Critical for financial data, medical records, backups, archives,
- How to Improve:
  Store multiple copies of data
  Distribute copies across zones and regions

Availability vs Durability

Concept	Focus	Metric Example
Availability	Data is accessible now	99.99% (4 9's) uptime
Durability	Data is never lost	99.999999999% (11 9's)

database horizontal scaling

What is Consistency?

- Scenario: You update your data. Should that update be visible in all replicas immediately, or is it okay if some replicas take a few seconds to catch up?
- Goal: Choose the right consistency model based on the balance between speed and data accuracy.

database horizontal scaling

Examples of Consistency Models

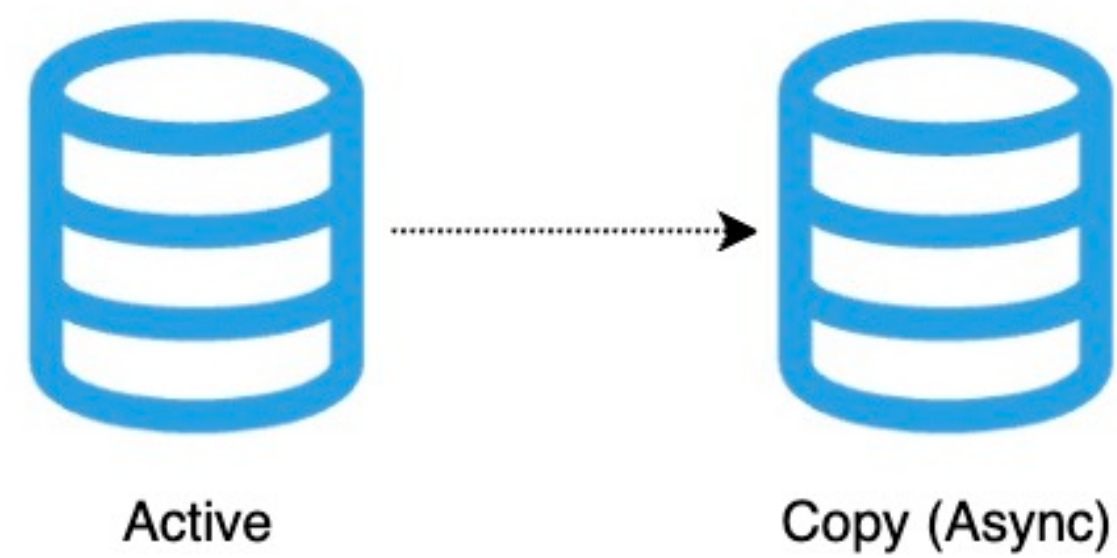
1. Strong Consistency

- Definition: All replicas return the same, most recent value after a write
- How It Works: Synchronous replication — write must complete in all replicas before confirming success
- Use Case: Banking transactions, inventory systems
- Trade-offs:

  High integrity, but slower performance

2. Eventual Consistency

- Definition: All replicas will eventually reflect the latest value — but may show different values for a short period
- How It Works: Asynchronous replication — write completes in one node, and propagates later
- Use Case: Social media posts, product reviews, user feeds
- Trade-offs:
  - High performance and scalability, but temporary inconsistency
  - Suitable when speed is more important than immediate accuracy



database async

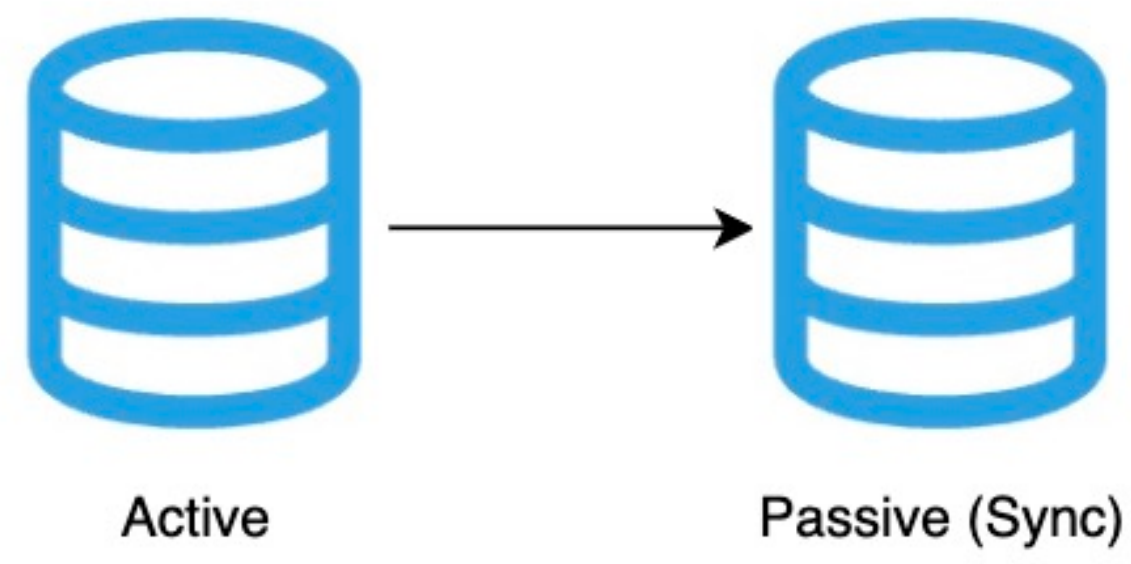
Choose the right database based on your Consistency requirement

Consistency Model
Strong Consistency
Eventual Consistency

Q2 Compare RTO vs RPO

RTO vs RPO – What's the Difference?

- Scenario: If a system crashes or data is lost, your business wants to know: How soon can we recover? and How much data can we afford to lose?
- Goal: Understand and define Recovery Time Objective (RTO) and Recovery Point Objective (RPO) for your applications



database sync

RTO (Recovery Time Objective)

- Definition: Maximum acceptable downtime after a failure
- Focus: How quickly can you restore service?
- Example:

  RTO = 1 hour System must be back online within 1 hour

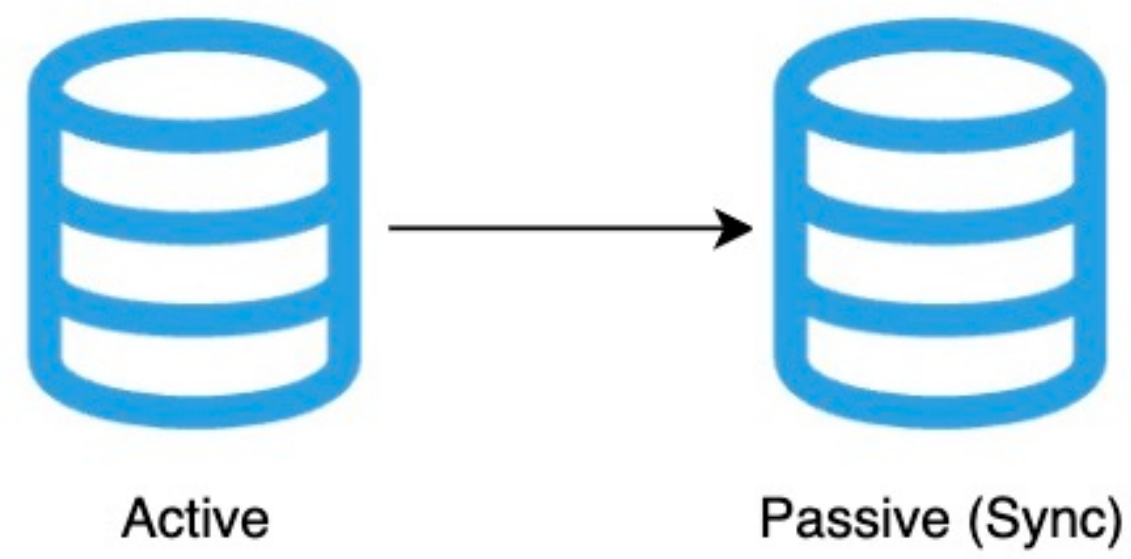
RPO (Recovery Point Objective)

- Definition: Maximum acceptable data loss (measured in time)
- Focus: How much data can you afford to lose?
- Example:

  RPO = 10 minutes You can tolerate losing 10 minutes of data

RTO vs RPO

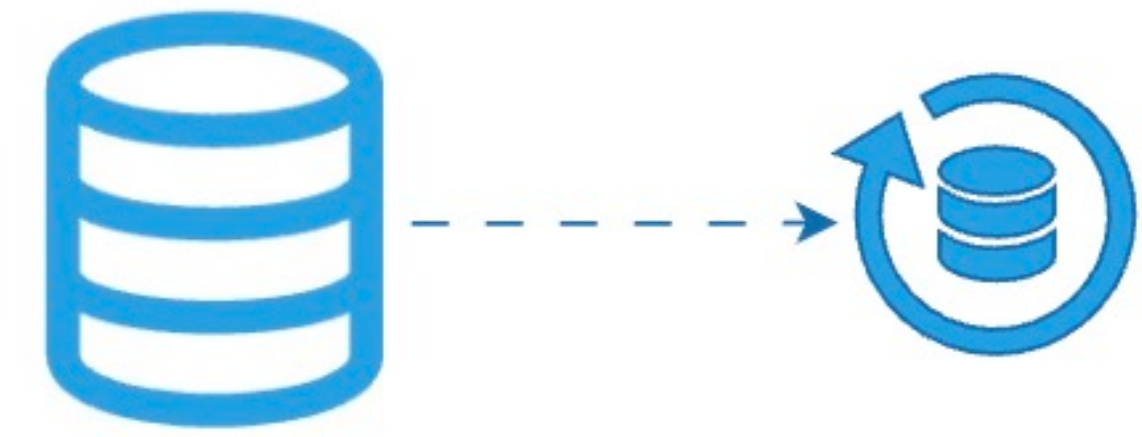
Metric	What It Measures	Example
RTO	Max downtime allowed	Recover in 1 hour
RPO	Max data loss allowed	Lose max 10 minutes of data



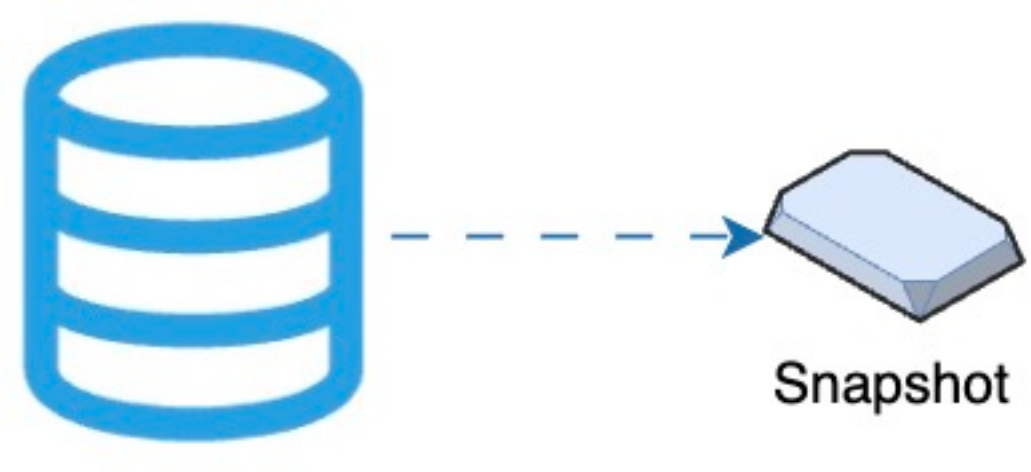
database sync

> Choosing the Right Strategy

RTO/RPO Requirement
High RTO + High RPO
Medium RTO/RPO
Low RTO + Low RPO



database backup

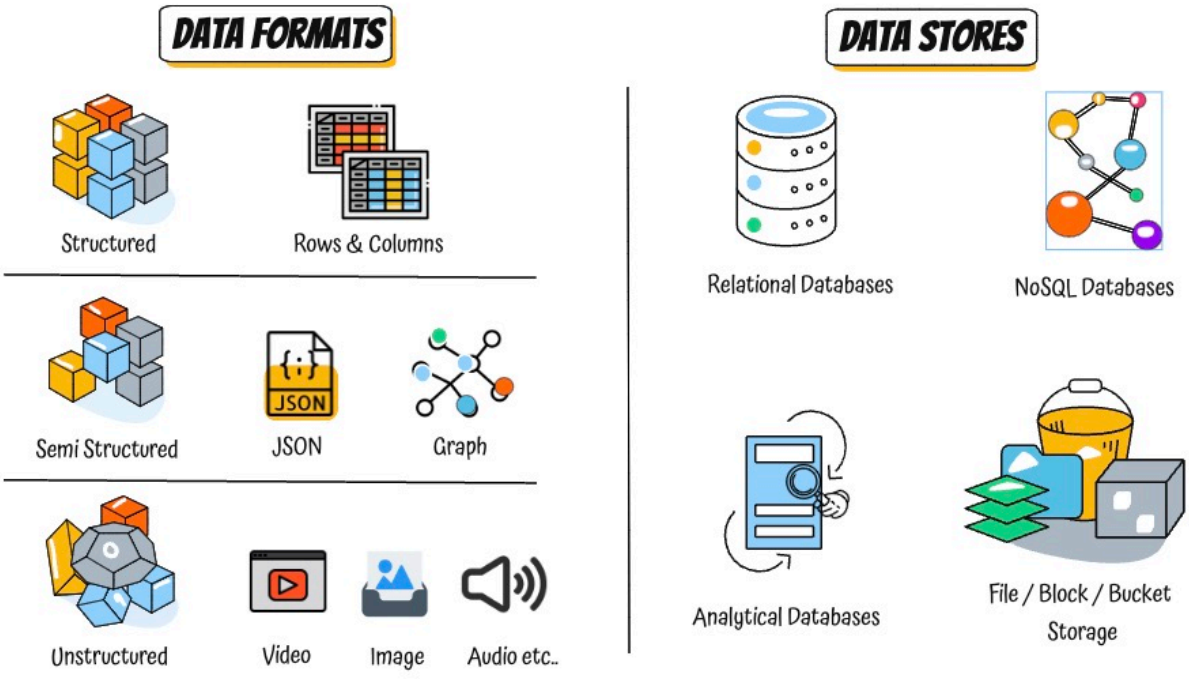


database snapshot

Q3 Compare Data Formats vs Data Stores

Types of Data Formats

- Structured: Tables, rows, and columns — Typically used by Relational databases (e.g., bank records)
- Semi-Structured: JSON, XML, key-value pairs — Typically used by NoSQL databases
- Unstructured: Files like images, audio, video, PDFs



data formats

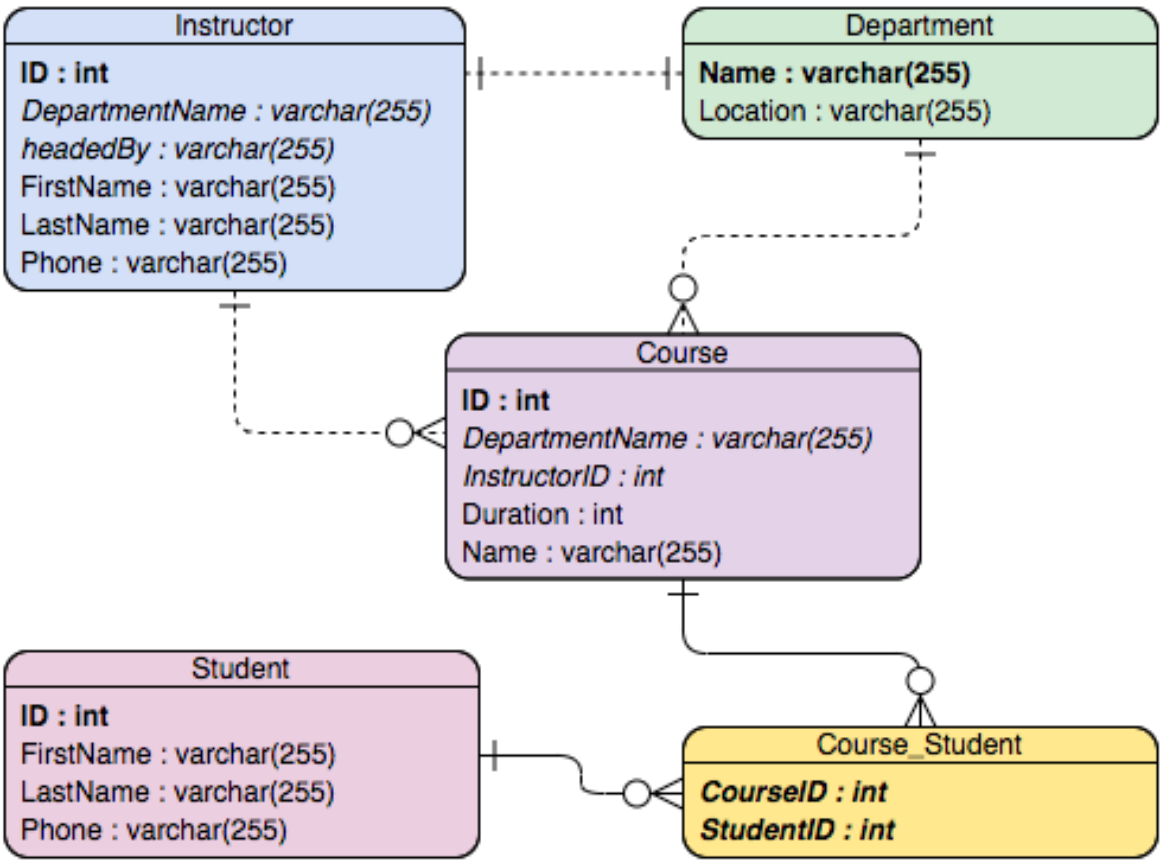
Types of Data Stores

- Definition: Services that store, manage, and retrieve different types of data
- Types:
  - Relational Databases – Structured transactional data
  - NoSQL Databases – Flexible document, key-value, or graph data
  - Analytical Databases – Petabyte-scale analytics and reporting
  - Object/Block/File Storage – Used for backups, media, and other unstructured data

Which Data Format for which Data Store?

- Structured: Relational and Analytical
- Semi-Structured: NoSQL
- Unstructured: Object/Block/File Storage

Q4 Where is Structured Data stored? OLAP vs OLTP?



database relational

Relational Databases (Structured)

- Tables with Rows & Columns: Fixed schema, strict relationships
- Used in
  - OLTP: Fast reads/writes for high-volume transactions
  - OLAP: Analytical queries over massive data (stored in columnar format)

OLTP (Online Transaction Processing):

- Small, frequent transactions – Ex: Transfer money
- Examples:
  - Banking system – money transfers, balance checks
  - E-commerce – order placement, payment updates
  - Reservation systems – booking tickets, seat updates

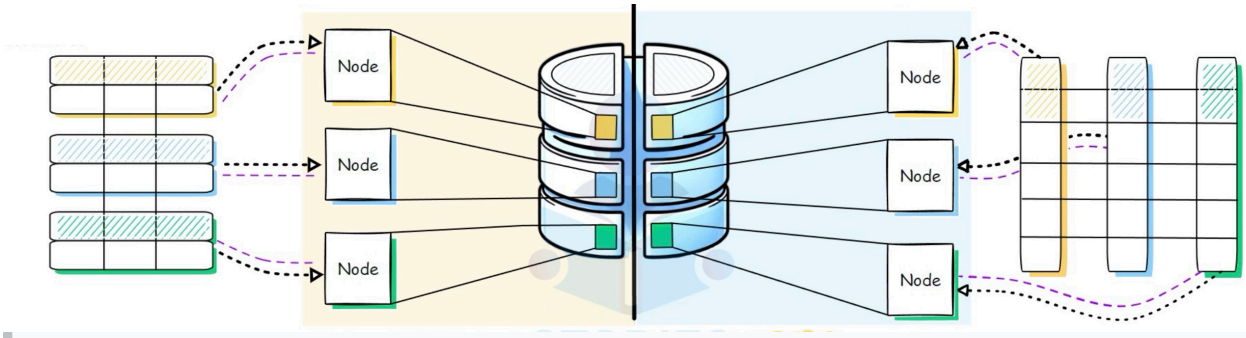
OLAP (Online Analytical Processing):

- Run complex queries on historical data
- Examples:
  - Sales trends by region over months
  - Customer behavior analytics
  - Financial reporting and dashboards

OLTP vs OLAP

Aspect	**OLTP**
Purpose	Day-to-day transactions
Query Type	Short, simple, real-time queries
Examples	Bank transfers, orders, bookings
Terminology	Transactional Databases
Data Structure	Row-based
Typical Architecture	One Large Node with standby (Some modern databases are Distributed)

database relational column row



Relational Databases in the Cloud

Type	AWS	Google Cloud
OLTP	Amazon Relational Database Service, Amazon Aurora	Cloud SQL, Cloud Spanner
OLAP	Amazon Redshift	BigQuery

Global vs Regional Relational OLTP Databases

Feature	Global Database
Definition	A database deployed across multiple regions, offering low-latency global access and automatic replication
Availability	Very High – Resilient to regional failures
Cost	Higher due to multi-region storage and replication
Example – AWS	Amazon Aurora Global Database
Example – Azure	Azure SQL with geo-replication (creates a continuously synchronized, readable secondary database)
Example – Google Cloud	Cloud Spanner (multi-region instance)

Q5 Where is Semi-Structured Data stored? (NoSQL Databases)

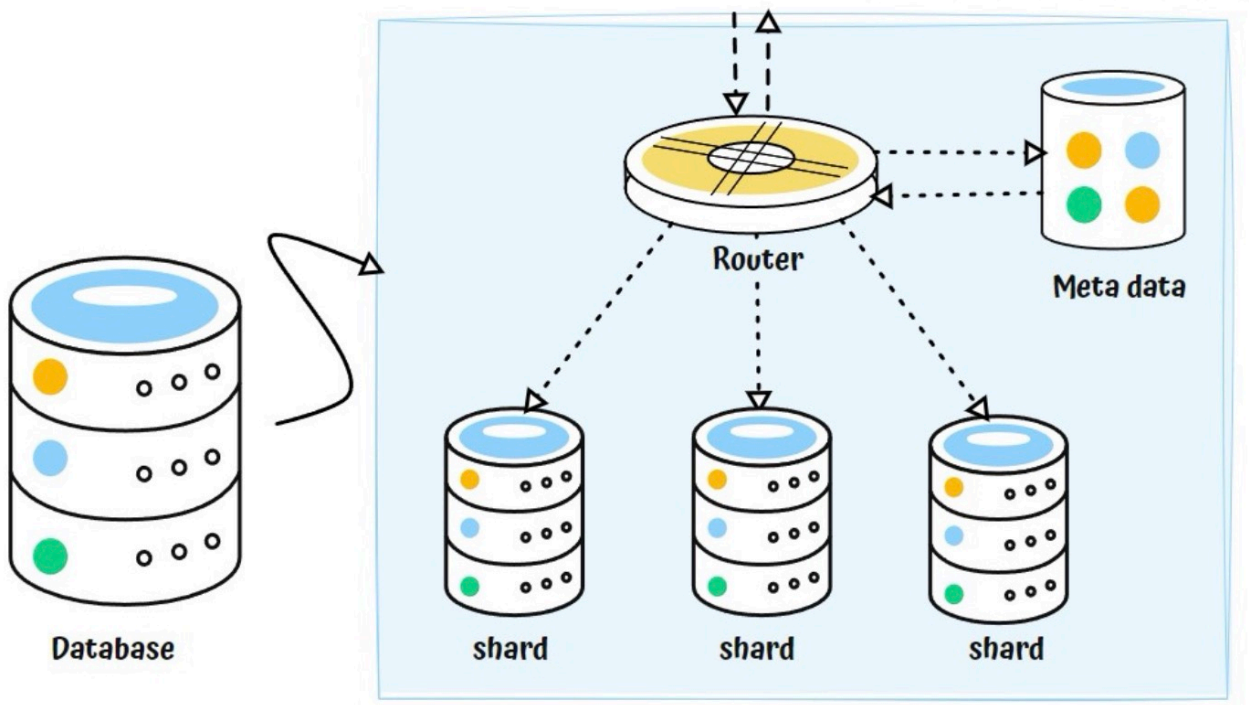
Why Semi-Structured Data?

- Scenario: Imagine building a product catalog or user profile where each record has different fields — some users have a Twitter handle, others do not
- Need: A flexible format that can evolve with your application without changing the database schema every time
- Definition: Data with some structure but not rigid like relational tables
- Examples: JSON documents, key-value pairs, graphs
- Use Cases: Profiles, product catalogs, ..

data semi structured

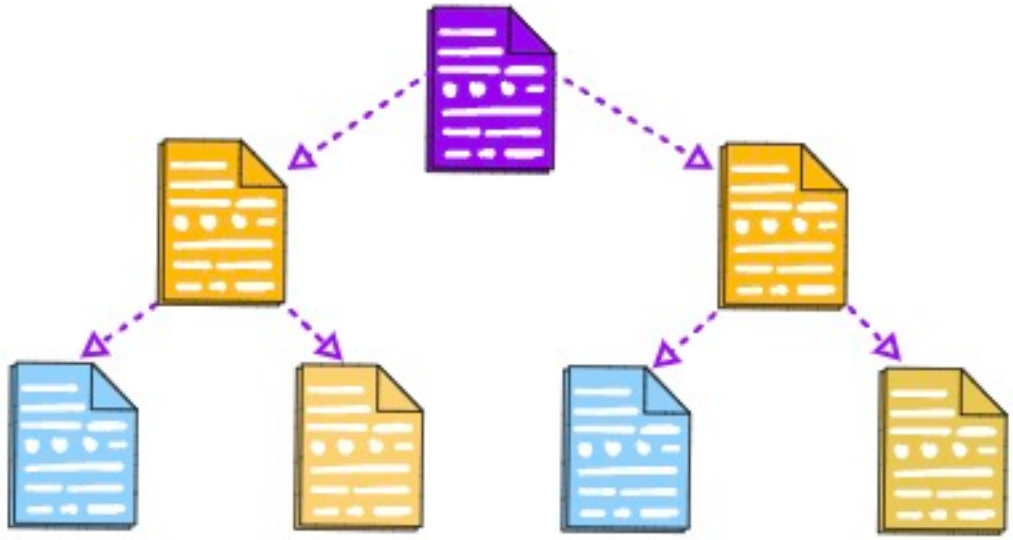
NoSQL Databases are used to store Semi-Structured Data

- NoSQL = Not Only SQL: Flexible schema, high performance, and horizontal scalability
- Designed For: Massive scale and rapid changes in data format
- Adaptable: App controls the schema instead of the database



database scaling horizontal

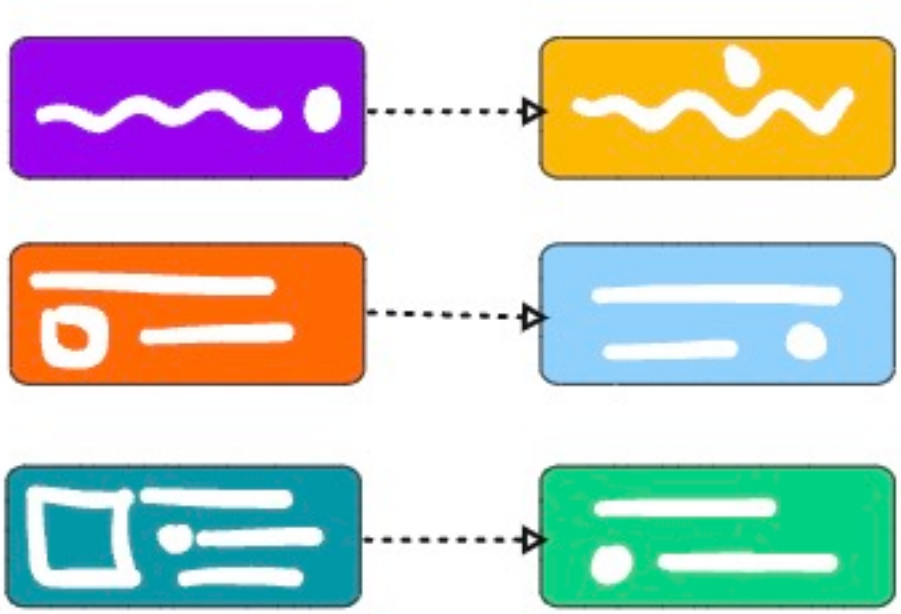
> Important NoSQL Database Types
>
> - 1: Document Databases
> - 2: Key-Value Databases
> - 3: Graph Databases
> - 4: Column-Family Databases
>
> 1: Document Databases



database nosql document

> - Data stored as JSON-like documents
> - Each document has a unique key
> - Structure can vary from document to document
> - Use Cases: Shopping cart, user profile, product catalog
> - Managed Services: Amazon DynamoDB, Amazon DocumentDB , Azure Cosmos DB (SQL API), Google Cloud Firestore
>
> 2: Key-Value Databases

database nosql key value



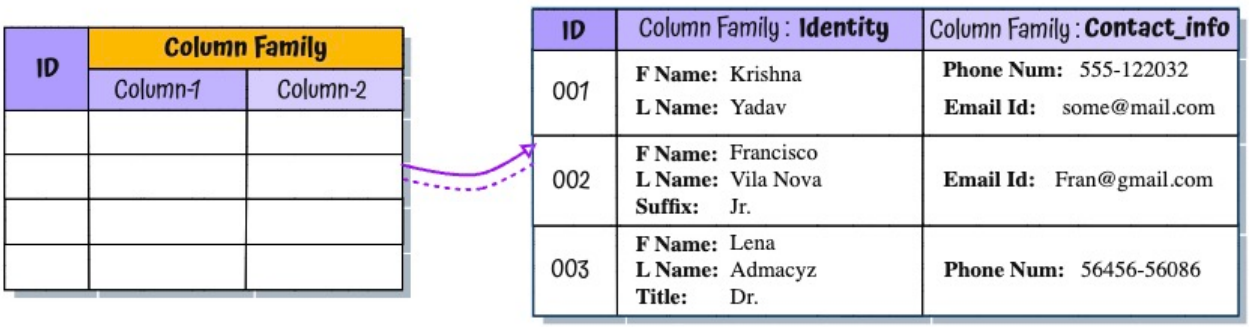
- Data stored as a key and its corresponding value
- Very fast lookups by key
- Use Cases: Caching, session management
- Managed Services: Amazon DynamoDB, Azure Cosmos DB (Table API), Google Cloud Firestore

3: Graph Databases

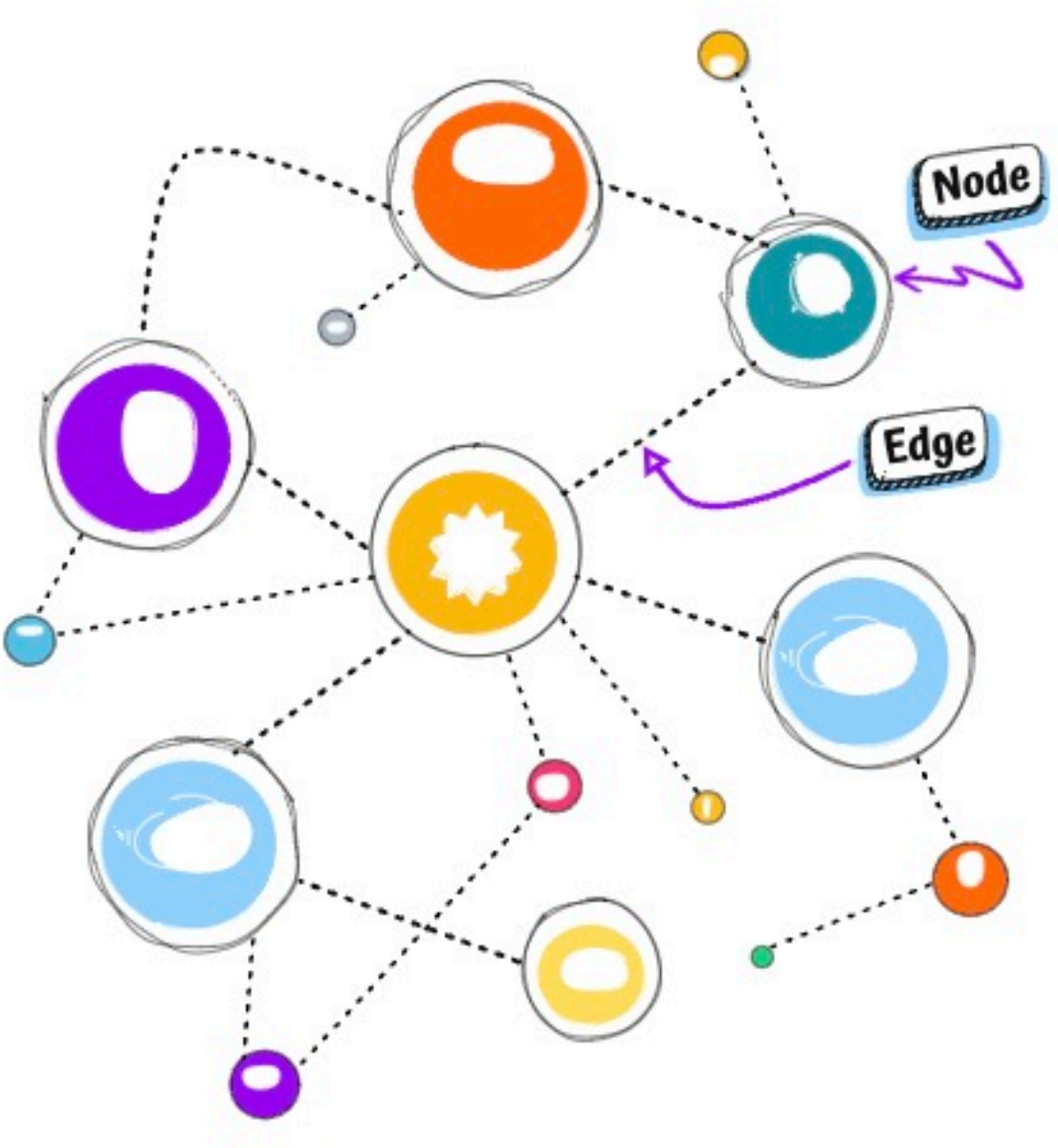
database nosql graph

- Data modeled using nodes and relationships (edges)
- Great for capturing and querying complex relationships
- Use Cases: Social networks, recommendation engines, fraud detection
- Managed Services: Amazon Neptune, Azure Cosmos DB Gremlin API, Spanner Graph

4: Column-Family Databases



database nosql columnar



- Data stored in rows and columns grouped into families
- Sparse format – rows don't need to have all columns
- Use Cases: IoT data, time-series data, analytics
- Managed Services: Amazon Keyspaces (Cassandra), Azure Cosmos DB Cassandra API, Google Cloud Bigtable

Example

Code

```
"user_profiles": [
  {
    "id": 101,
    "name": "Alice",
    "email": "[email protected]",
    "twitter": "@alice_dev"
  },
  {
    "id": 102,
    "name": "Bob",
    "email": "[email protected]"
    // Bob has no Twitter handle
  }
]

"product_catalog": [
  {
    "id": "A1",
    "name": "Smartphone",
    "brand": "BrandX",
    "camera_specs": "12MP",
    "battery_life": "10h"
  },
  {
    "id": "B2",
    "name": "Laptop",
    "brand": "BrandY",
    "ram": "16GB",
    "storage": "512GB SSD"
    // No camera_specs for laptops
  }
]
```

Code

```
//session1
{
  "key": "abc123",
  "value": {
    "userId": "u001",
    "loginTime": "2050-07-24T10:00:00Z",
    "role": "admin"
  }
}

//session2
{
  "key": "xyz789",
  "value": {
    "userId": "u002",
    "loginTime": "2050-07-24T10:05:00Z",
    "role": "viewer"
  }
}
```

Code

```
{
  "nodes": [
    { "id": "u1", "name": "Ranga" },
    { "id": "u2", "name": "Ravi" },
    { "id": "u3", "name": "John" },
    { "id": "u4", "name": "Sathish" }
  ],
  "edges": [
    { "from": "u1", "to": "u2", "label": "FRIEND" },
    { "from": "u2", "to": "u3", "label": "FRIEND" },
    { "from": "u3", "to": "u4", "label": "FRIEND" },
    { "from": "u4", "to": "u1", "label": "FRIEND" }
  ]
}
```

Code

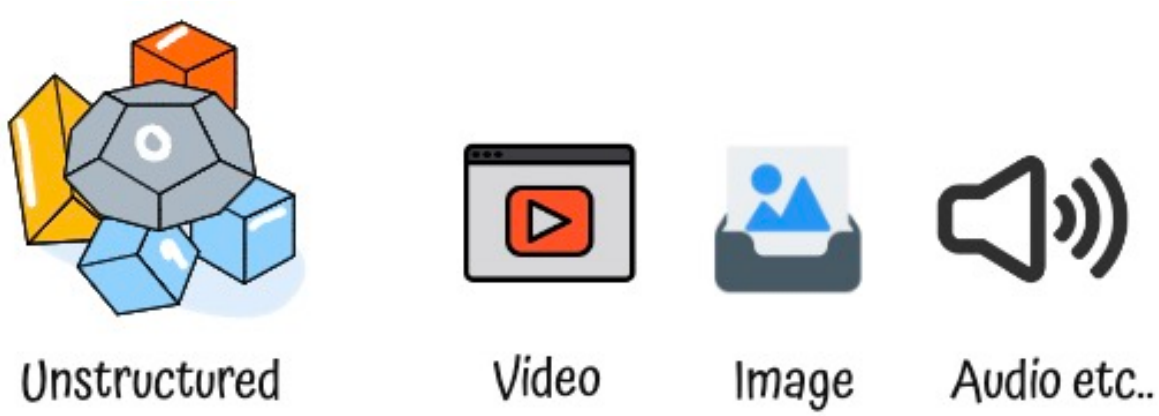
```
{
  // Unique identifier for the row
  // typically identifies a device, user, or service
  "rowKey": "device123",

  "columnFamilies": {

    // First column family stores time-based log entries
    "logs": {
      // Timestamp as column name, log message as value
      "2050-07-24T10:00:00Z": "Temperature: 32°C",
      "2050-07-24T10:01:00Z": "Temperature: 33°C",
      "2050-07-24T10:02:00Z": "Temperature: 34°C"
    },

    // Second column family stores system statuses
    "status": {
      // Same timestamp as column name, status message as value
      "2050-07-24T10:00:00Z": "OK",
      "2050-07-24T10:01:00Z": "OK",
      "2050-07-24T10:02:00Z": "ALERT: Temp threshold exceeded"
    }
  }
}
```

Q6 Where is Unstructured Data stored? (File, Block, or Object storage)

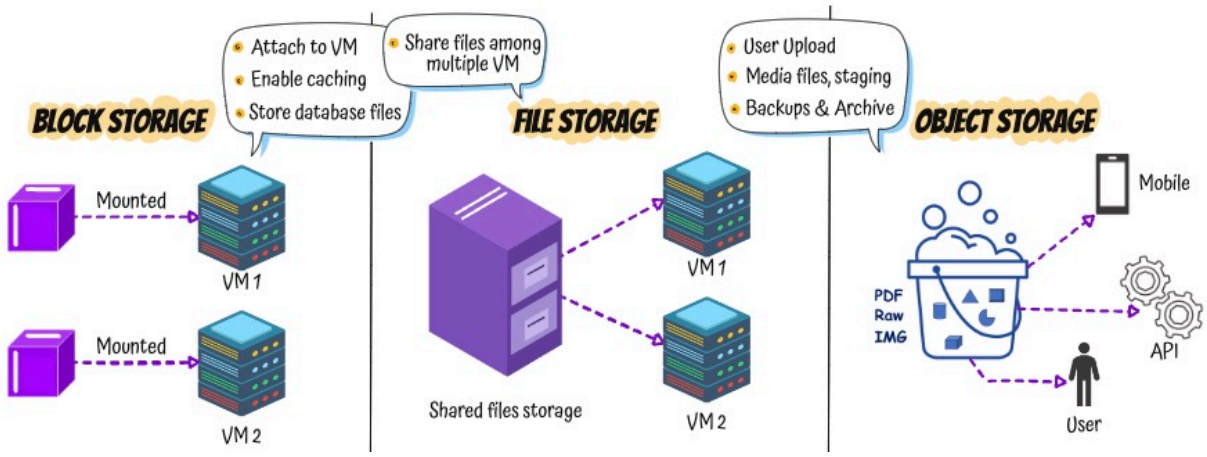
Why Unstructured Data?

- Scenario: Imagine building YouTube — videos, thumbnails, subtitles, and logs. All this content doesn't fit into a table.
- Need: A way to store and retrieve large files like videos, images, documents, and logs — without predefined structure.
- Definition: Data without a fixed schema or format (e.g., audio, video, PDFs, images, binaries)
- Examples: Uploaded files, media content, backup archives, sensor logs
- Handled Using: File, block, or object storage based on access pattern and use case

data unstructured



> Types of Storage for Unstructured Data
>
> - 1: Block Storage
> - 2: File Storage
> - 3: Object Storage

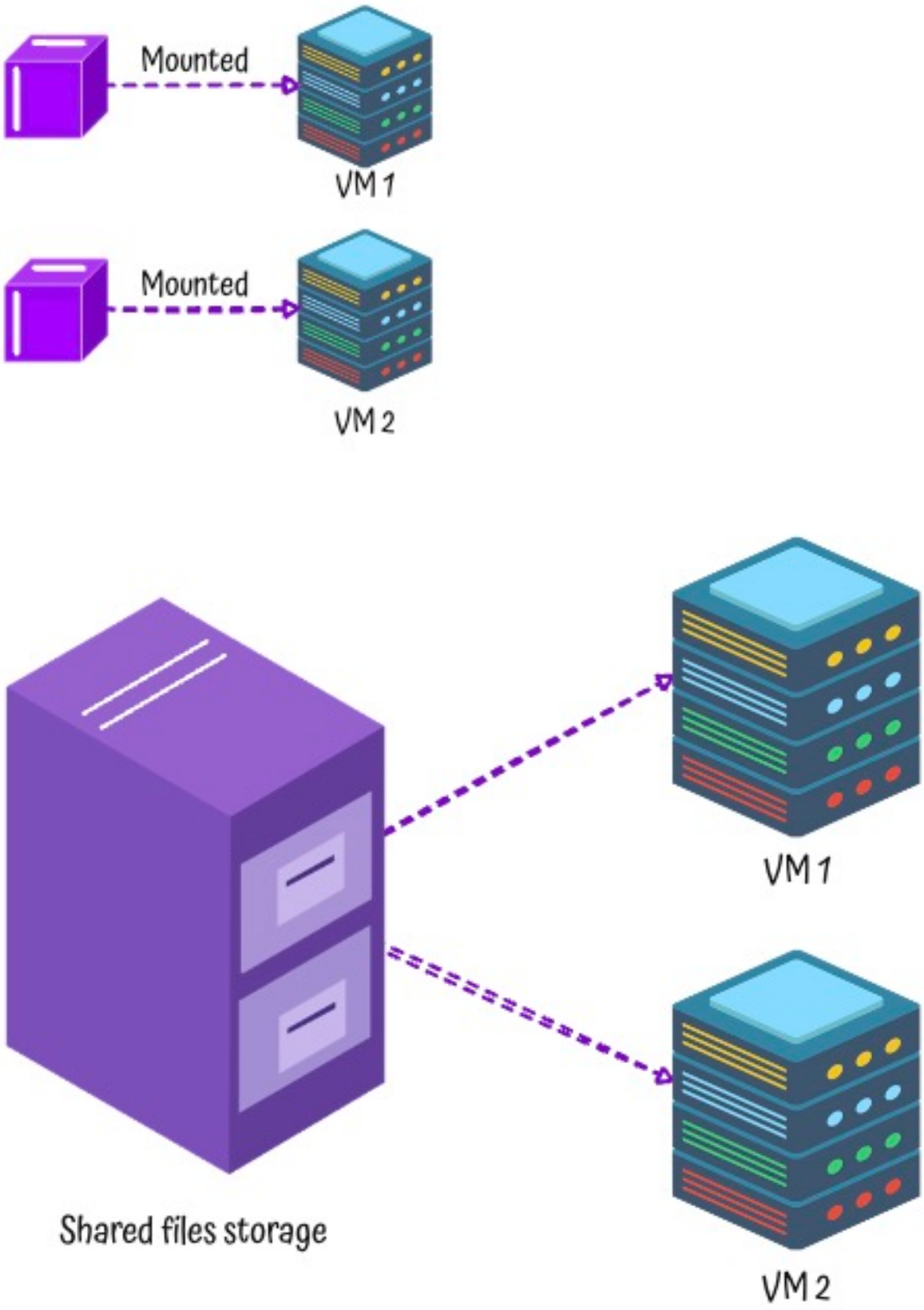


storage types

> 1: Block Storage

storage block

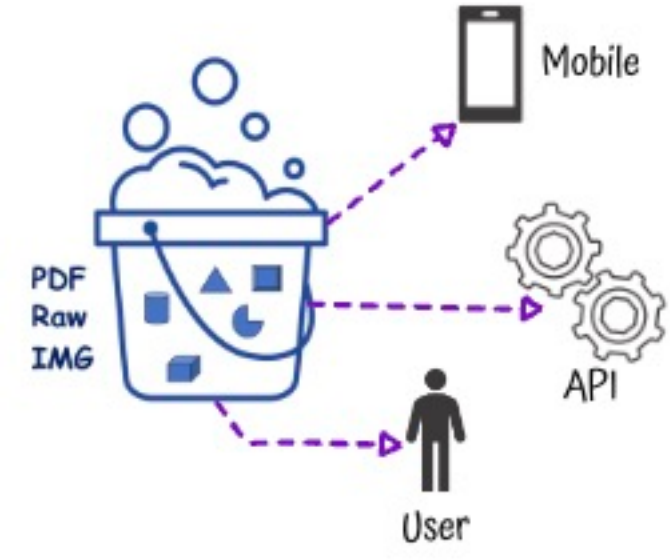
> - Low-level storage used like a hard disk
> - High performance for structured workloads (e.g., VM disks, databases)
> - Use Cases: OS disks, DB volumes
>
> 2: File Storage



storage file

- Shared file systems accessed over network using file paths
- Ideal for applications needing traditional file structure and shared access
- Use Cases: Team file shares, CMS systems

3: Object Storage



storage object

- Data stored as objects (data + metadata + unique ID)
- Accessed using REST APIs — no mounting required
- Scalable, cost-effective, and durable
- Use Cases: Media hosting, backups, logs, static websites

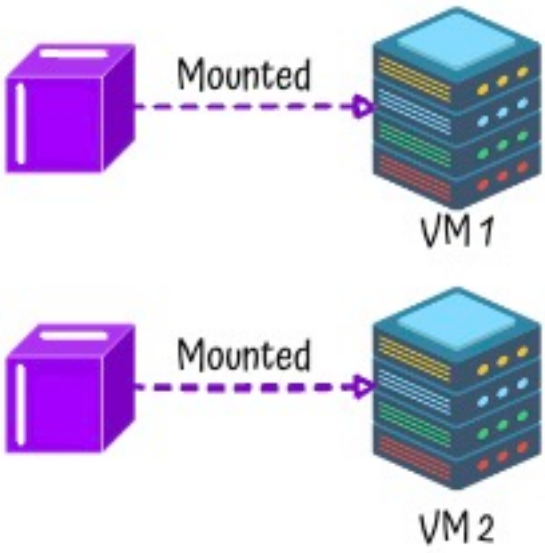
Q7 What is Block Storage?

Why Block Storage in Cloud?

- Scenario: Imagine running a virtual machine or a database in the cloud. You need a reliable, fast disk that behaves like a physical hard drive.
- Block Storage: Provides raw storage volumes that can be attached to servers and used just like local disks.
- Goal: Attach virtual disks to compute resources like VMs, containers, and databases

storage block

Key Characteristics



- Raw Volumes: You format and mount it like a traditional disk
- Detachable and Reusable: Can be attached, detached, and re-attached to different servers
- Persistent: Data stays intact even if the VM is stopped or restarted

Use Cases

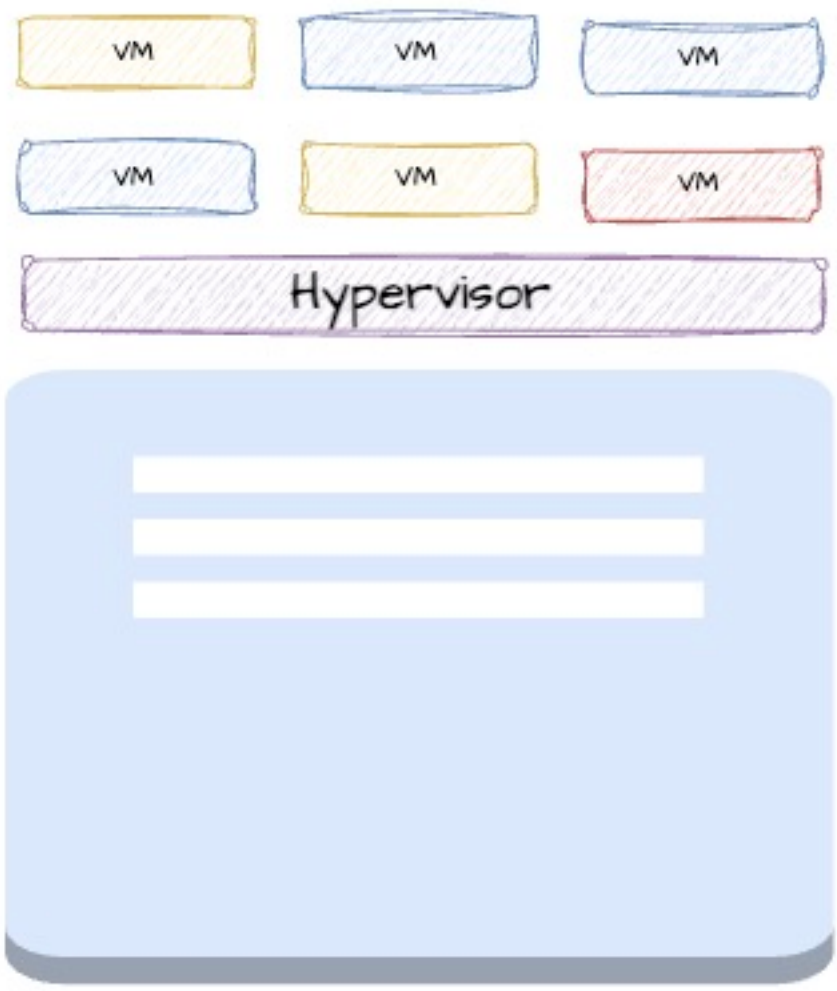
- Virtual Machines: OS and data disks for cloud-based servers
- Databases: High-speed transactional storage for SQL and NoSQL engines

virtual machines

Choice 1: Types of Block Storage

- Persistent Block Storage (e.g., Network Attached like EBS)
  - Stored separately and connected over the network
  - Retains data across VM stops, starts, or replacements
  - Ideal for critical data like databases or file systems
- Temporary Block Storage (e.g., Instance Store)
  - Physically attached to the VM host
  - Very fast but data is lost if VM is terminated
  - Best for temporary data like cache or scratch files

Choice 2: HDD (Hard Disk Drive) vs SSD (Solid State Drive)



- HDD (Hard Disk Drive)
  - Transactional Performance: Lower – slower for frequent reads/writes
  - Throughput: High – good for sequential access of large files
  - Strength: Best for large, sequential data processing
  - Use Cases:
  - Big data workloads
  - Log processing
  - Backup or archival
  - Cost: Lower – budget-friendly
- SSD (Solid State Drive)
  - Transactional Performance: High – excellent for fast, frequent access
  - Throughput: High – handles both small and large data well
  - Strength: Great for small, random and sequential I/O
  - Use Cases:
  - Databases
  - Web servers
  - Operating system volumes

Cost: Higher – but justified for high performance

Cloud Managed Services for Block Storage

- AWS: Amazon EBS, Instance Store
- Azure: Azure Managed Disks, Temporary Disks
- Google Cloud: Persistent Disks, Local SSDs

Q8 What is File Storage?

Why File Storage?

- Scenario: Imagine a team working on shared documents, code files, or project reports. Everyone needs access to the same files, organized in folders.
- File Storage: A storage system that organizes data in a familiar folder and file format, accessible over a shared network.

storage file

What is File Storage?

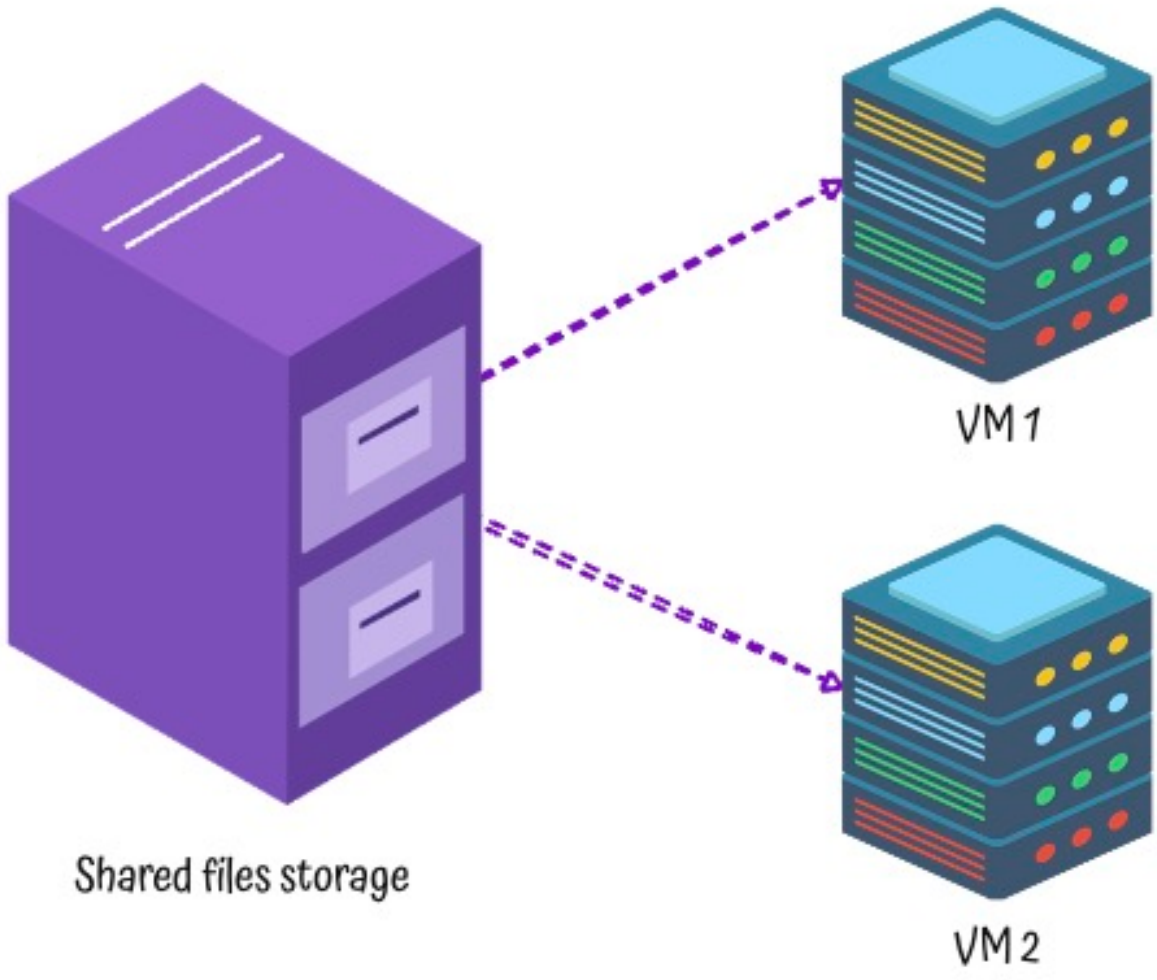
- Definition: A way to store data in hierarchical structure – folders and files
- Goal: Share files easily between users or systems
- Access Method: Files accessed using standard protocols like NFS or SMB
  NFS: A protocol designed to share files over a network in Linux/Unix systems
  SMB: A protocol used mainly in Windows environments to share files

How File Storage Works

- Mounted Volumes: File storage is mounted like a network drive
- Shared Access: Multiple users or servers can read/write files
- Folders & Files: Organize data just like on your laptop

Benefits of File Storage

- Easy to Use: Familiar structure – folders, paths, extensions
- Shared Access: Ideal for collaboration
- Reliable: Supports backups, snapshots, and versioning

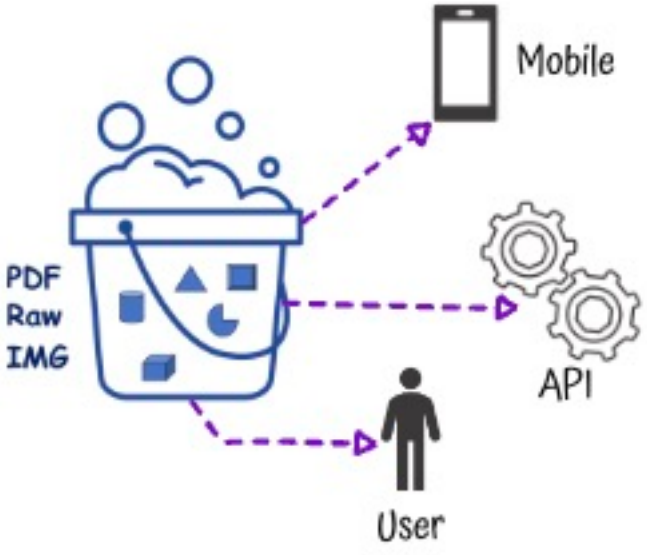


> Cloud Managed Services for File Storage
>
> - AWS: Amazon EFS (Elastic File System), Amazon FSx
> - Azure: Azure Files
> - Google Cloud: Filestore

Q9 What is Object Storage?

storage object

> Why Object Storage in Cloud?



- Scenario: Imagine millions of users uploading photos, videos, and documents through a website or mobile app — and accessing them anytime, from anywhere. Traditional file systems struggle to scale, manage access, or serve global traffic efficiently.
- Object Storage: Designed to store and retrieve large volumes of unstructured data (like media and backups) with high durability, scalability, and global accessibility.
- Goal: Store and retrieve any type of file at scale using a simple API

Key Characteristics

- Flat Structure: No folders — everything is stored in a bucket with a unique key
- Scalable: Can handle billions of files without performance loss
- Durable: Data is automatically replicated across multiple zones or regions
- Access via HTTP APIs: Easy to integrate with applications, websites, and mobile apps

Use Cases

- Backup and Archiving: Reliable, long-term storage
- Web Content: Store and serve images, videos, documents
- Big Data and Analytics: Ingest large files for processing
- Application Storage: Store logs, exports, reports

Choice 1: Versioning

- Purpose: Keep multiple versions of an object to protect against accidental deletes or overwrites
- Use Case: Restore a previous version of a file or track changes over time

Choice 2: Storage Tiers/Storage Class

- Purpose: Store data in the most cost-effective tier based on how often it is accessed
- Balance Performance and Price: Pay more for speed when needed, save more when data is rarely used
- Example Tiers
  - Standard or Hot: For frequently accessed data
  - Infrequent Access or Cool: For data accessed less often
  - Archive: For long-term storage with slower retrieval

Choice 3: Lifecycle Rules

- Purpose: Automatically transition or delete objects based on age or other characteristics
- Use Case: Move old files to cheaper storage (e.g., archive) or delete them after a set period to save costs

Cloud Managed Services for Object Storage

- AWS: Amazon S3 (Standard, IA, Glacier, , ..)
- Azure: Azure Blob Storage (Hot, Cool, Archive , ..)
- Google Cloud: Cloud Storage (Standard, Nearline, Coldline, Archive, ..)

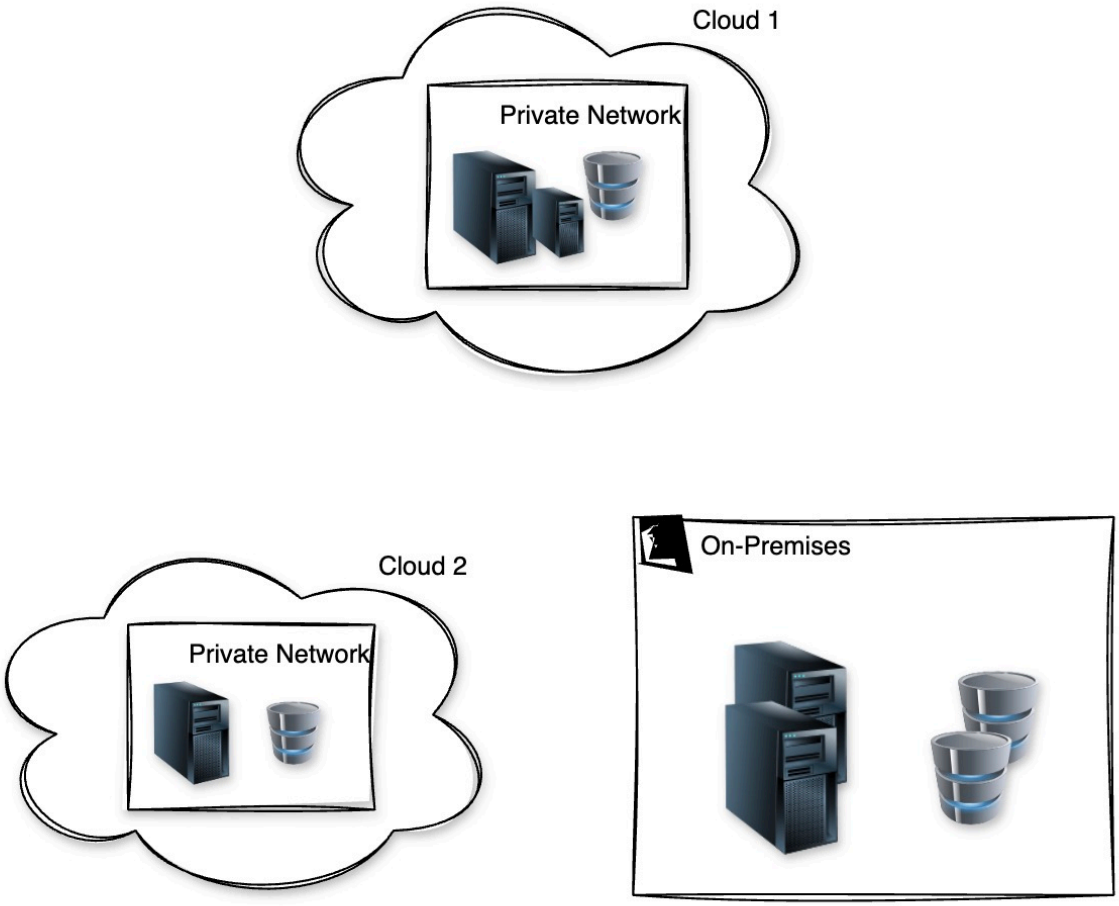
Interesting Thing to Know

- Amazon Glacier Service: Amazon Glacier was launched as a standalone service for long-term storage with very low cost.
- Optimized for Archiving: Great for backups, compliance data, and archived content that can wait minutes or hours to be retrieved.
- Overtime Integrated into S3: Part of Amazon S3 as a storage class
- Simple Management: Use S3 interface to store data in Glacier – no need to manage a new service.
- All Clouds Offer Archive Storage as a Storage Class/Tier: AWS (Glacier), Azure (Archive), GCP (Coldline, Archive)

Q10 What is Hybrid Storage?

Why Hybrid Storage?

- Scenario: Imagine storing large datasets which need to be accessed on-premises — some frequently accessed, some rarely touched. Keeping all of it in the cloud may be slow. Keeping it all on-prem may limit scalability.
- Hybrid Storage: Combines on-premises and cloud storage to balance cost, speed, and control.



hybrid cloud

What is Hybrid Storage?

- Definition: A storage solution that bridges local (on-premises) storage with cloud storage
- Goal: Enable seamless data access and movement between on-prem and cloud environments
- Use Cases:
  - Gradual cloud migration
  - Burst storage for peak workloads
  - Archive and backup to cloud

Benefits of Hybrid Storage

- Scalability: Cloud extends your capacity without new hardware
- Cost Efficiency: Store only hot data locally, cold data in the cloud
- Performance: Local access for critical data
- Data Protection: Cloud backup improves disaster recovery

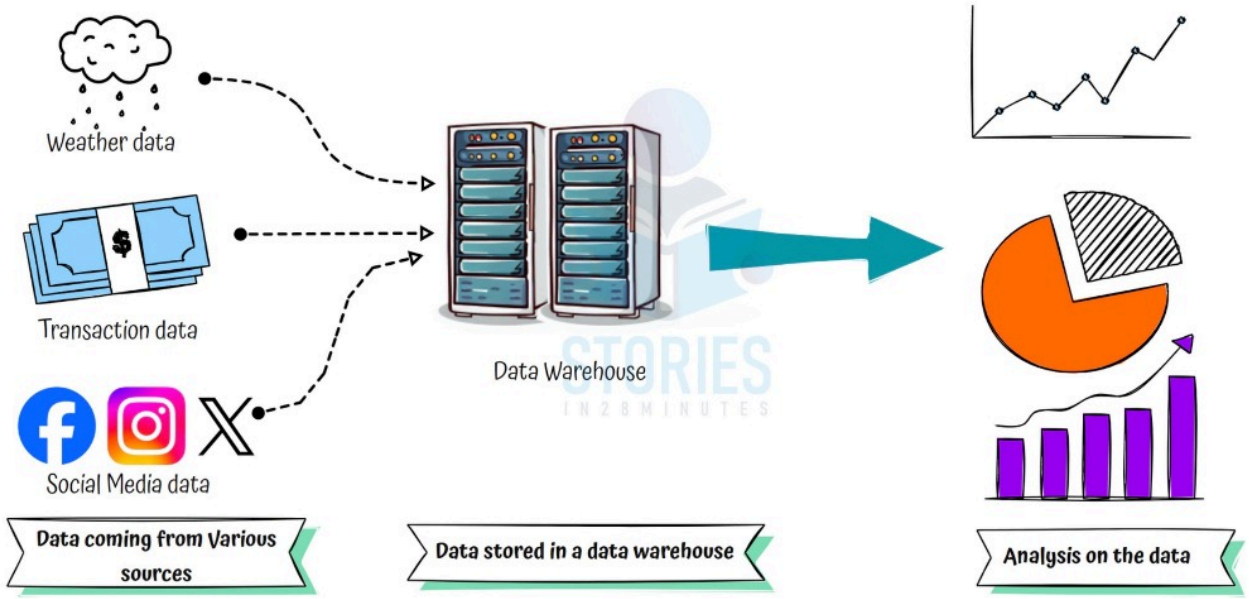
Cloud Managed Services for Hybrid Storage

- AWS: AWS Storage Gateway
- Azure: Azure File Sync
- Google Cloud: Google Cloud Filestore (with Hybrid Connectivity)

Q11 What is the need for Data Analytics?

Why Data Analytics?

- Scenario: You have tons of raw data — purchases, transactions, sensor readings. But without analysis, it's just noise.
- Data Analytics: The process of analyzing raw data to extract useful insights
- Goal: Make data-driven decisions to improve business outcomes



data analytics warehouse

Data Sources Can Include

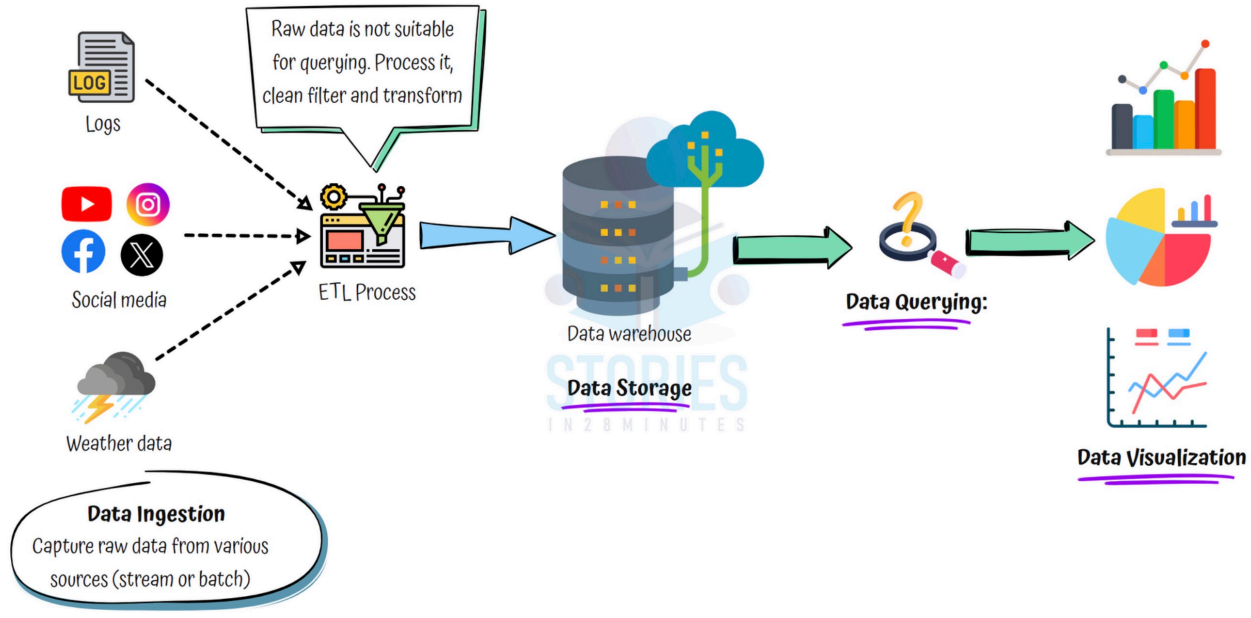
- Transactions: Purchases, payments, logs
- Sensor & IoT Data: Temperature, pressure, GPS
- External Feeds: Weather, stock prices, social media
- Internal Systems: CRM, HR, finance systems

Key Benefits of Data Analytics

- Uncover Trends: Understand customer behavior, market shifts, and product performance
- Identify Weaknesses: Spot bottlenecks, inefficiencies, or risks early
- Improve Outcomes: Enhance efficiency, customer satisfaction, and profitability

Why Data Analytics Workflow?

- Scenario: You collect tons of raw data from multiple sources. But raw data is not useful until it's cleaned, processed, and visualized.
- Workflow: Follows a clear step-by-step path from raw data to business insights.



data analytics process

Data Ingestion

- Goal: Collect raw data from multiple sources
- Sources: Websites, IoT sensors, apps, logs, transactions
- Modes:
  - Batch: Data loaded periodically
  - Stream: Data ingested in real-time (e.g. user clicks, weather sensors)

Data Processing

- Goal: Make data usable for analysis
- Clean: Remove duplicates and errors
- Filter: Eliminate irrelevant or outlier data
- Transform: Convert to a consistent format or structure
- Aggregate: Combine data for summaries or insights

Data Storage

- Where: Store in a data warehouse
- Goal: Centralize data for easy access and future analysis

Data Querying

- What: Run SQL-like queries to analyze trends, identify patterns

Data Visualization

- Why: Charts and dashboards make insights easier to understand
- Impact: Helps leadership spot trends, outliers, and make informed decisions

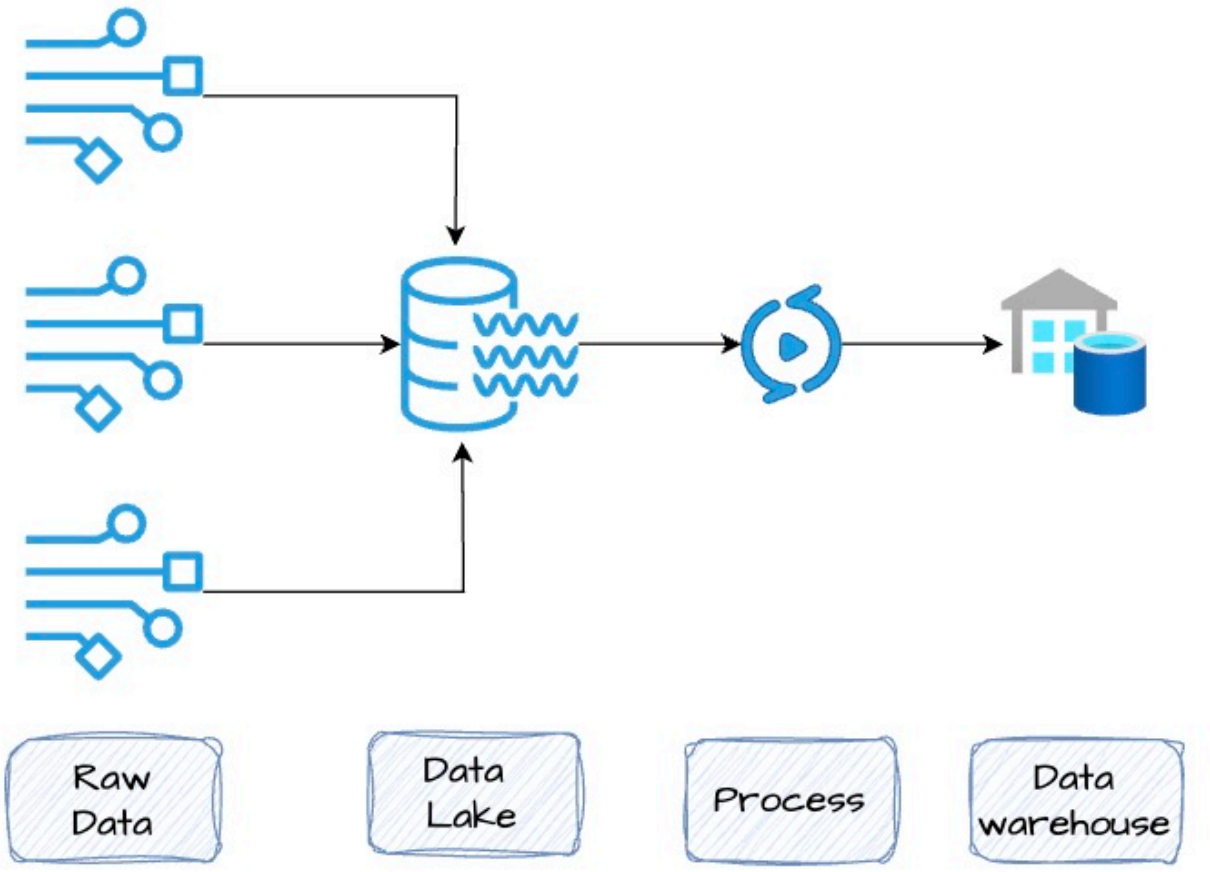
Cloud Managed Services for Data Analytics

Stage	AWS	Azure
Streaming Ingestion	Amazon Kinesis	Azure Event Hubs
Data Processing (ETL / Data Prep)	AWS Glue	Azure Data Factory
Data Warehouse and Querying	Amazon Redshift	Azure Synapse Analytics
Data Visualization	Amazon QuickSight	Power BI

Q12 Compare Data Warehouse vs Data Lake

The 3Vs of Big Data

- Volume: Massive datasets — from terabytes to petabytes to exabytes
- Variety: Mix of structured (tables), semi-structured (JSON), and unstructured (videos, logs)
- Velocity: Speed of data arrival — batch (hourly/daily) or real-time (streams)



data analytics datalake

What if the Data We're NOT Capturing Becomes Valuable Later?

- Businesses may miss future insights if they only store processed data
- Storing raw data now gives the flexibility to run AI, ML, or new analytics later
- A Data Lake solves this — store everything today, use what you need tomorrow

Data Warehouse vs Data Lake

- Data Warehouse
  - Stores processed, structured data optimized for fast SQL queries
  - Used for business intelligence, dashboards, and reporting
  - Examples: Teradata, Amazon Redshift, Google BigQuery, Azure Synapse Analytics
- Data Lake
  - Stores raw, unprocessed data — compressed and cost-efficient
  - Can handle any format (CSV, JSON, images, audio, logs, etc.)
  - Supports on-demand exploration, AI/ML, and analytics workflows
  - Built on object storage
  - Examples: Amazon S3, Google Cloud Storage, Azure Data Lake Storage Gen2

How They Work Together

- Data Lake holds everything — raw logs, events, images, clickstreams, ...
- Data Warehouse pulls from the lake — after processing
- Modern Tools: Services like Google BigQuery, Azure Synapse Analytics, and Amazon Athena can query data directly from the data lake — no need to move or duplicate

Cloud Managed Services for Big Data Storage and Analytics

- AWS:
  - Storage: Amazon S3
  - Warehouse: Amazon Redshift
  - Query-over-lake: Amazon Athena
- Azure:
  - Storage: Azure Data Lake Storage Gen2
  - Warehouse: Azure Synapse Analytics
  - Query-over-lake: Synapse Serverless SQL
- Google Cloud:
  - Storage: Google Cloud Storage
  - Warehouse: BigQuery
  - Query-over-lake: BigQuery External Tables

CHAPTER 3

Asynchronous Communication in the Cloud

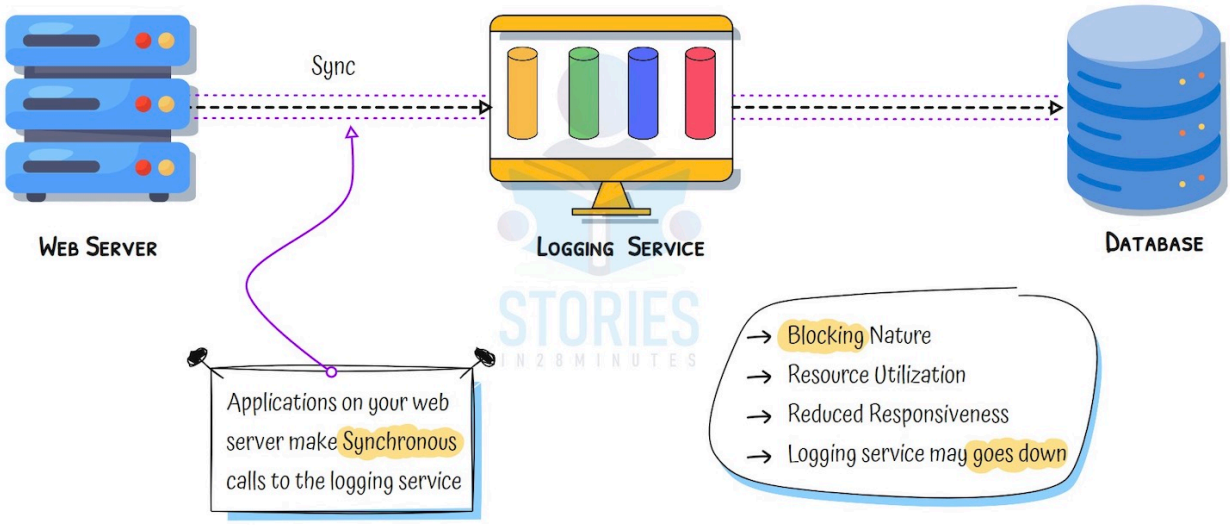
6 Questions & Answers

3 Asynchronous Communication in the Cloud

Q1 Why Asynchronous Communication?

Synchronous Communication – Example

- Scenario: A web app calls a logging service to store logs for every user action
- Tight Coupling: The web app waits for the logging service to respond before continuing
- Failure Risk: If the logging service is slow or crashes, the user request may fail or be delayed
- Latency Impact: Every dependency adds to response time, affecting user experience



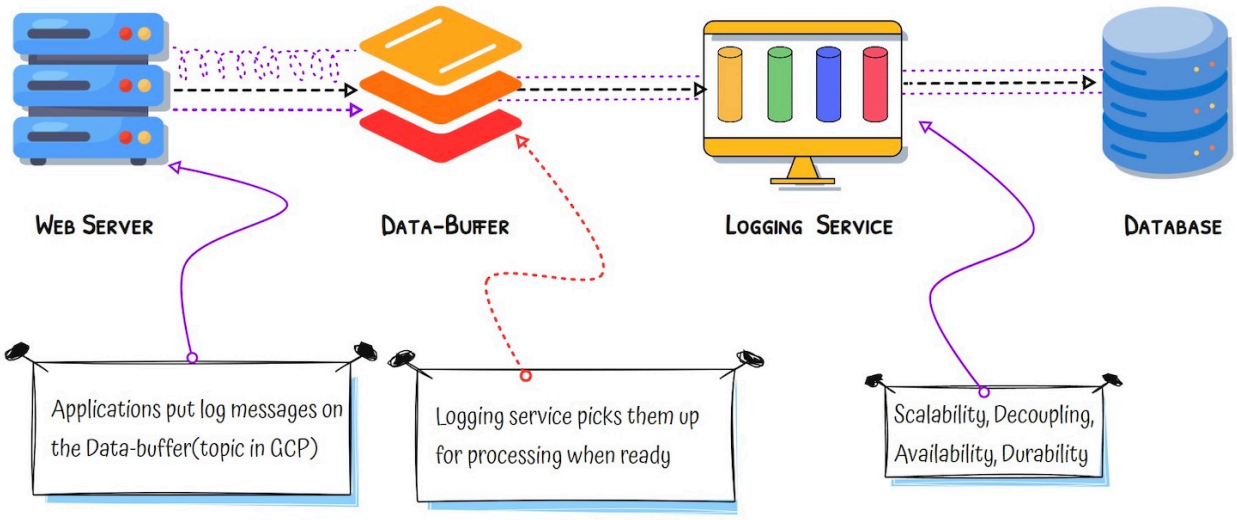
communication sync

Asynchronous Communication – Decoupled Systems

- What is Changed?: The web app sends logs to a message queue instead of the logging service directly
- Queue in the Middle: Messages are safely stored and processed later
- Loose Coupling: The app does not need to wait — it moves on instantly
- Resilience: If the logging service is down, messages are still stored and processed later
- Traffic Spikes Handled Gracefully: Queue absorbs spikes; services can catch up later

communication async

Synchronous vs Asynchronous Communication



Synchronous
Tight coupling between systems
Fails if downstream service fails
Harder to scale during spikes
Real-time but fragile
Use when real-time response is essential – e.g., payment confirmation, authentication, ..

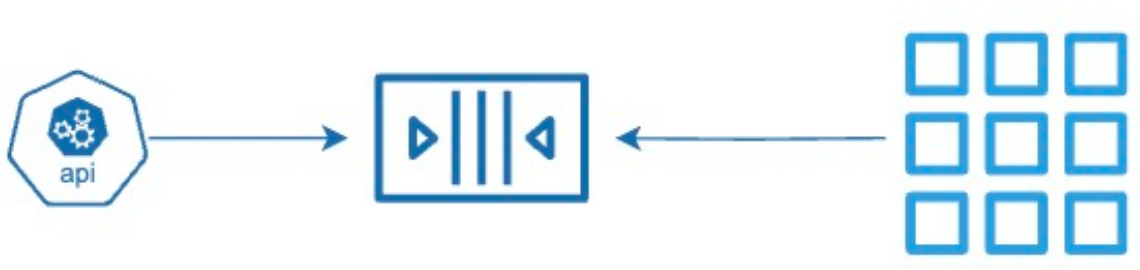
> Patterns in Asynchronous Communication
>
> Note: These patterns are not mutually exclusive. A system might use an event bus to trigger a workflow, which pulls from a queue and publishes updates via pub/sub.

Pattern
Queue-based Messaging
Publish/Subscribe
Streaming Data
Event Bus
Workflow Orchestration

Q2 What is Queue-based Messaging (Pull)?

Why Queue-based Messaging?

- Scenario: Imagine an online store that takes orders quickly, but the delivery system processes them slowly — both work at different speeds
- Queue-based Messaging: A temporary holding system where messages (like orders) wait until the next system is ready to process them. Helps decouple services and smooth out spikes in traffic.



async pull

Step By Step

- Step 01: Create a message queue to temporarily store messages between systems that don’t run at the same speed
- Step 02: A producer sends a message to the queue — for example, placing an order or uploading a task
- Step 03: The message is safely stored in the queue until the consumer is ready — this avoids dropping messages during high load
- Step 04: A consumer reads the message from the queue when it is ready — one at a time or in batches
- Step 05: After processing, the consumer can delete the message from the queue to prevent duplicate work
- Step 06: If the consumer fails to process the message, it stays in the queue or is moved to a dead-letter queue for review
- Step 07: Multiple consumers can be added to scale processing without changing the producer
- Reliable and Decoupled: The producer and consumer do not need to run at the same time — the queue makes the system more resilient and scalable

Key Benefits

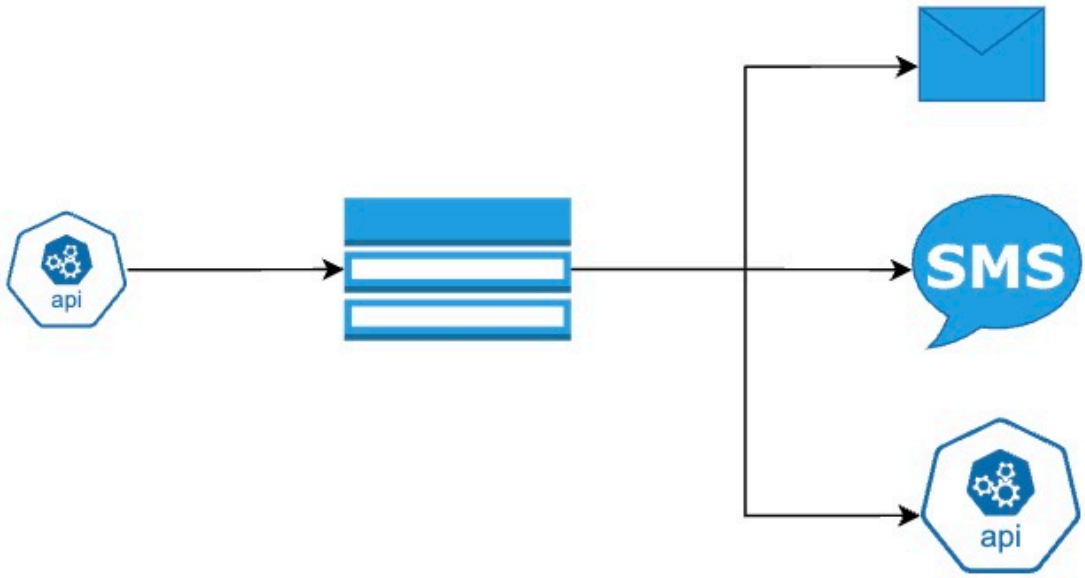
- Decoupled Systems: No need to run at the same time
- Pull-Based: Consumers read messages when ready
- Reliable Delivery: Messages are stored safely
- Scalable: Scale producers and consumers separately
- Error Handling: Use retries and dead letter queues for failures

Queue Service
Amazon SQS
Azure Queue Storage
Google Cloud Pub/Sub (Pull)

Q3 What is Publish Subscribe Messaging (Push)?

Why Publish/Subscribe?

- Scenario: Imagine a photo-sharing app where uploading a photo should notify multiple services — one for resizing, one for tagging, one for storing metadata
- Publish/Subscribe: A messaging pattern where one event is sent to multiple subscribers at the same time. Each system gets its own copy and processes it independently
- Goal: Notify multiple systems at once



async push

Step By Step

- Step 01: Create a topic to act as a central channel where messages will be published
- Step 02: One or more subscribers register their interest in receiving messages from this topic
- Step 03: A publisher sends a message to the topic — for example, a new user signs up or a file is uploaded
- Step 04: The topic immediately forwards the message to all subscribers — this is a push-based system
- Step 05: Each subscriber gets a separate copy of the message and can handle it independently
- Step 06: Subscribers can be of different types — webhooks, email, SMS, queues, or even functions
- Step 07: If a subscriber is temporarily unavailable, the system may retry or store the message depending on the setup

Key Benefits

- Fan-Out: One message, many receivers
- Push-Based: Delivered instantly — no polling
- Loosely Coupled: Publisher doesn't know subscribers
- Flexible Targets: Works with queues, APIs, emails, and more
- Great for Notifications: Trigger multiple actions in parallel

Cloud Service	Provider
Amazon SNS	AWS
Azure Event Grid	Azure
Google Cloud Pub/Sub	Google Cloud

Q4 What is the need for Streaming Data?

Why Streaming Data?

- Scenario: Imagine a website where users click links, watch videos, and interact every second. You want to track all of it in real time for analytics or alerts
- Streaming Data: Data that arrives continuously in small chunks, usually with a timestamp
- Goal: Process data instantly as it arrives
- Example: Track user clicks on a website in real time

async streaming data

Step By Step

- Step 01: Set up a streaming service that can collect and process data in real time — one event at a time
- Step 02: A producer (like a mobile app, sensor, or server) continuously sends data to the stream — such as logs, clicks, or metrics
- Step 03: The streaming platform partitions the data and stores it temporarily for processing
- Step 04: One or more stream processors read from the stream, often in real time or near real time
- Step 05: The processor may clean, transform, filter, or enrich the data before passing it on
- Step 06: The processed data can be sent to multiple destinations — like dashboards, storage, or alerts
- Step 07: The system keeps moving — new data keeps flowing in, and the pipeline keeps processing without pause

Key Benefits

- Real-Time Input: Handles logs, metrics, clicks as they happen
- Ordered & Timestamped: Events come in sequence with time info
- Instant Processing: Power dashboards, alerts, fraud detection
- Scalable: Stream split into partitions for parallel processing

Cloud Service	Provider
Amazon Kinesis Data Streams	AWS
Azure Event Hubs	Azure
Google Cloud Pub/Sub (Streaming)	Google Cloud
Amazon MSK / Azure Kafka / Google Cloud Kafka	All

Q5 Why do we need an Event Bus?

What Are Cloud Service Events?

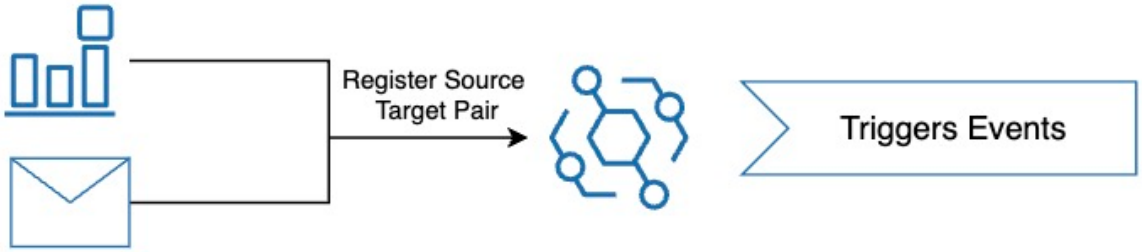
- Definition: Notifications automatically generated by cloud services when something happens (e.g., file uploaded, resource deleted)
- Goal: Automate reactions to changes in your cloud environment
- Examples:
  - Object added to a storage bucket
  - New VM instance created
  - API Gateway receives a request
  - Database row updated

Common Use Cases

- Storage: Process or scan files after upload
- Security: Alert when IAM policies change
- Automation: Start pipelines when code is committed
- Compliance: Log and archive data when resources are modified

Why do we need an Event Bus?

- Scenario: You need to process a Cloud Service Event. Imagine a virtual machine (VM) is stopped — and you want to automatically trigger a serverless function to send an alert, log the event, or clean up temporary resources.
- Event Bus: You can register the VM service as the event source, and the serverless function as the target. When the VM is stopped, the event bus detects the event and automatically routes it to the function — without writing any polling logic or custom glue code.
- Centralized Routing: Event Bus works with cloud service events, partner events, and custom events from your own applications — all routed through a central system that applies rules to determine where each event should go



async event bus

Step By Step

- Step 01: Set up an event-driven system that reacts to cloud resource changes — like file uploads to storage
- Step 02: Define an event source (e.g., an object storage bucket) and configure a target (e.g., serverless function, queue, or notification) that should handle the event
- Step 03: A user or system uploads a file to the storage bucket — this is the event trigger
- Step 04: The cloud platform automatically detects the event and forwards it to the configured target — no need to poll or check manually
- Step 05: The target processes the event — for example, a function resizes an image, extracts metadata, or sends an alert
- Step 06: The system can apply filters to trigger only when certain conditions are met
- Step 07: This works reliably and scales automatically — even if thousands of events occur per second
- Responsive and Serverless: Events trigger logic immediately, enabling real-time automation without managing infrastructure

Key Things to Know

- Trigger-Based Logic: Target is invoked in the event happens
- Real-Time Response: No polling or delay — events are captured and acted on instantly
- Flexible Targets: Events can trigger functions, queues, or workflows
- Filterable Events: Process only the events that matter using rules or filters

Supported in Different Cloud Platforms

Cloud Service	Provider
Amazon EventBridge	AWS
Azure Event Grid	Azure
Google Eventarc	Google Cloud

Why CloudEvents?

- Scenario: Imagine integrating events from different systems — each has a different format, causing confusion and complex code
- CloudEvents: A standard format for describing event data, making it easier to build event-driven systems across platforms
- Created by: CNCF (Cloud Native Computing Foundation)
- Goal: Solve the challenge of inconsistent event formats from different publishers

Key Advantages

- Consistency: Same format for all events — easier to read, parse, and debug
- Tooling Support: Use common libraries across clouds and languages (Java, Python, Go, etc.)
- Portability: Events can move between AWS, Azure, Google Cloud, or on-premises without rewriting code
- Interoperability: Producers and consumers can work independently

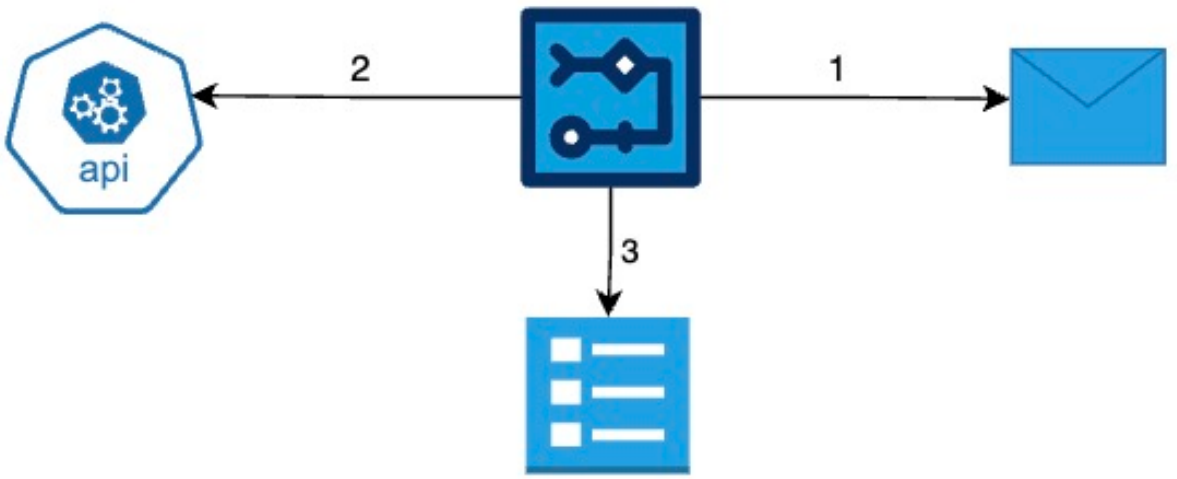
Cloud Managed Event Bus Services support CloudEvents

- AWS: Amazon EventBridge (supports CloudEvents format)
- Azure: Azure Event Grid (native support for CloudEvents v1.0)
- Google Cloud: Eventarc (uses CloudEvents as standard format)

Q6 What is the need for Workflow Orchestration?

Why Workflow Orchestration?

- Scenario: Imagine onboarding a new employee — it involves multiple steps like verifying documents, creating accounts, and sending welcome emails
- Workflow Orchestration: A system that coordinates all steps in the right order with logic, retries, and decision-making — and tracks the status from start to finish



async workflow

Step By Step

- Step 01: Define a workflow made up of multiple steps — each step performs a task like calling an API, running a function, or waiting for approval
- Step 02: Add decision logic — like if/else, retries, timeouts — to control the flow based on conditions or failures
- Step 03: Trigger the workflow from an event (such as a new order or new employee is created) or configure a schedule
- Step 04: The orchestrator ensures steps run in the right order, handles failures, and resumes from where it left off if needed
- Step 05: You can monitor, pause, or restart workflows
- Reliable and Visual: Automate multi-step processes across systems — with full visibility and no manual coordination

Key Things to Know

- Step-by-Step Automation: Coordinates tasks in a defined sequence
- Built-in Logic: Supports conditions, retries, and timeouts between steps
- State Tracking: Keeps track of which step succeeded or failed
- Works Across Systems: Connects APIs, functions, databases, and external systems
- Visual and Monitorable: Provides dashboards to monitor, audit, and debug workflows easily

Workflow Orchestration

Cloud Service	Provider
AWS Step Functions	AWS
Azure Logic Apps	Azure
Durable Functions	Azure
Google Workflows	Google Cloud

CHAPTER 4

Networking in the Cloud

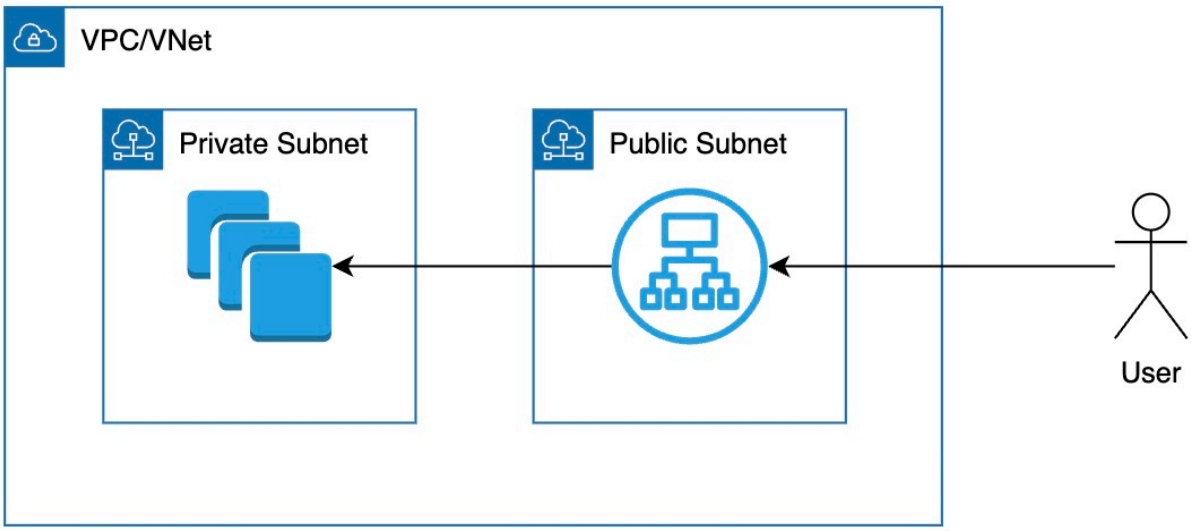
7 Questions & Answers

4 Networking in the Cloud

Q1 What is the purpose of Virtual Networks and Subnets in the Cloud?

What is a Virtual Network in the Cloud?

- Scenario: You're building a web application with a frontend, backend service, and a database — all hosted in the cloud. You want the frontend to talk to the backend, and the backend to talk to the database, but you don't want that internal communication to go over the public internet.
- Goal: Create your own private network in the cloud to keep internal traffic secure and isolated, and control access to it.



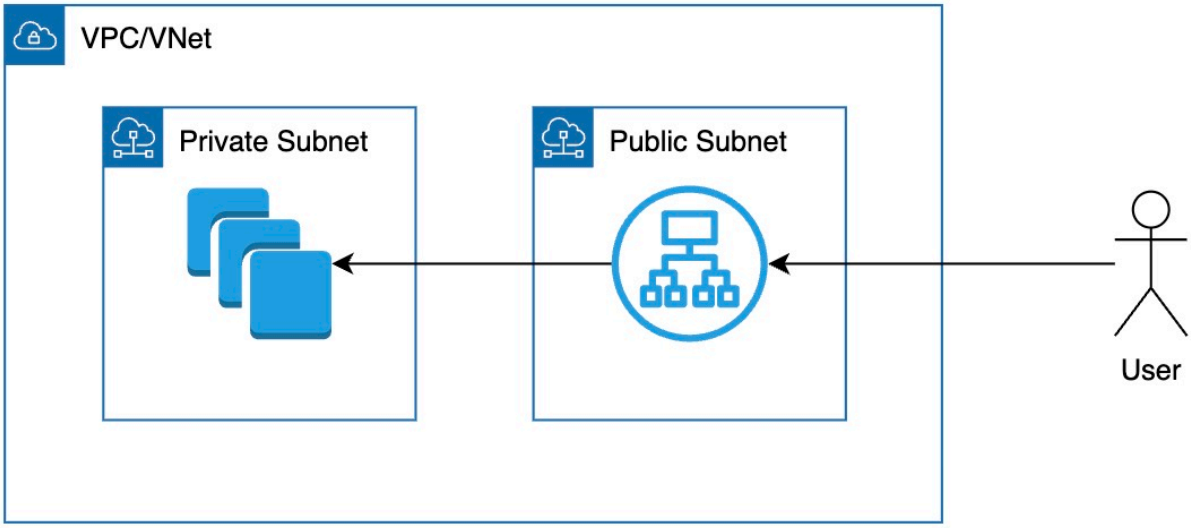
networking vpc subnet

Virtual Network (VNet / VPC)

- Definition: An isolated, configurable network environment in the cloud
- Purpose: Connect and secure cloud resources (e.g., VMs, storage, databases)
- Key Features:
  - Traffic Isolation: One virtual network is invisible to others
  - Control Inbound/Outbound Traffic: Control traffic to and from the network
  - Best Practice: Place all resources inside your virtual network

networking vpc subnet

Why do we need Subnets?



- Challenge: Different resources in your virtual network have different access requirements
  Example:
  Load balancers need to be accessible from the internet
  Databases should stay private and only be accessed internally
- Solution: Divide your virtual network into subnets
  Create logical segments to group similar types of resources
  Apply different rules to each subnet to control communication and improve security

Types of Subnets

Subnet Type	Access
Public Subnet	Accessible from internet
Private Subnet	Not directly accessible from internet

Cloud Managed Services for Virtual Networks

- AWS: VPC (Virtual Private Cloud), Subnets
- Azure: Virtual Network (VNet), Subnets
- Google Cloud: VPC Network, Subnets

Q2 What is the need for CIDR Blocks in Networking?

Why CIDR Blocks?

- Scenario: Imagine needing to assign different ranges of IP addresses to groups of resources in a structured way — like one range for databases, another for application servers.
- CIDR Blocks: Provide a way to define and group IP addresses efficiently using a prefix (e.g., 192.168.0.0/24), making network design easier.

What is a CIDR Block?

- Definition: Classless Inter-Domain Routing – a method to allocate IP addresses and define network sizes
- Format: IP address/prefix length (e.g., 10.0.0.0/16)
- Prefix Length: Tells how many bits are for the network (rest are for hosts)

CIDR Block Example

- 10.0.0.0/24:
  Represents IP addresses from 10.0.0.0 to 10.0.0.255
  Network Portion: First 24 bits Fixed (10.0.0)
  Host Portion: Last 8 bits 256 total IPs (254 usable)

Common CIDR Sizes

- /16: ~65,536 IPs – good for large networks
- /24: 256 IPs – common for subnets
- /32: 1 IP – used for a single host

CIDR in Cloud Networking

- VPC CIDR: Defines the overall IP space of a Virtual Private Cloud
- Subnet CIDRs: Divide the VPC CIDR into smaller ranges

Q3 Why are Gateways essential in cloud networking?

Why Gateways in Cloud?

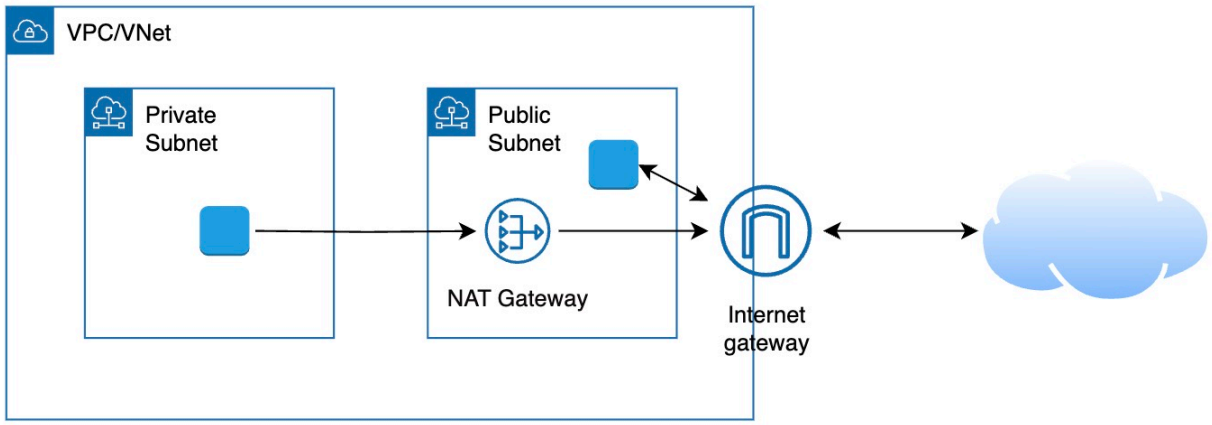
- Scenario: Your VM instance runs in a private network in the cloud. How does it connect to the internet or other networks securely?
- Gateways: Components that act as bridges between your cloud network and external networks like the internet, other cloud networks, or on-premises data centers.

What is a Gateway?

- Definition: A gateway connects different networks and manages the flow of traffic between them
- Goal: Enable communication while maintaining control and security

Internet Gateway

- Purpose: Enables resources inside your cloud network to communicate with the public internet. Without an internet gateway, virtual machines or load balancers in a public subnet can't be reached from the outside world.
- How it Works: When attached to a virtual network, the internet gateway handles both incoming and outgoing traffic for resources that have public IP addresses

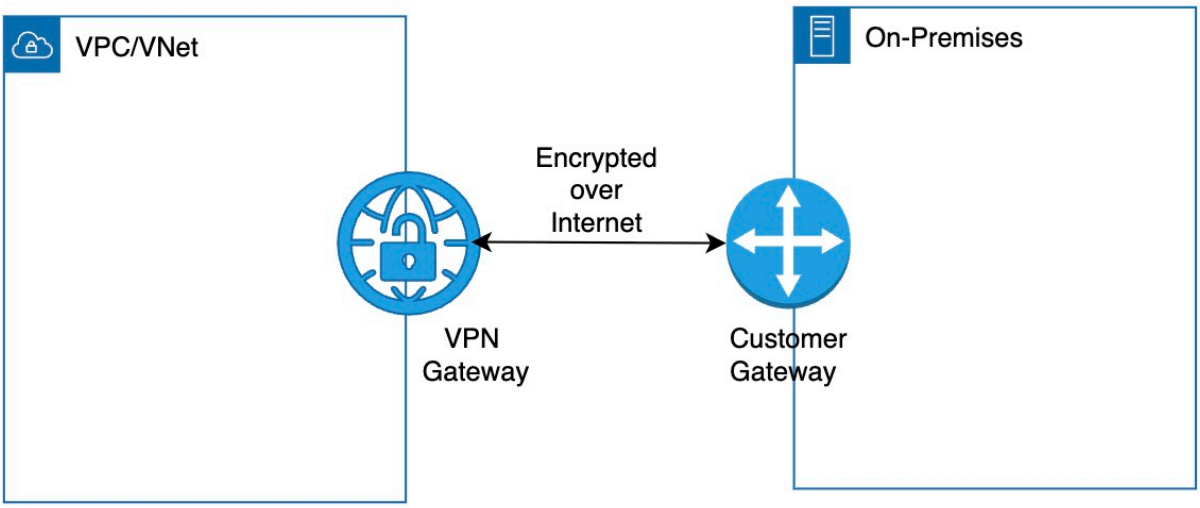


networking gateway

NAT Gateway

- Purpose: Allows resources inside private subnets to initiate outbound connections to the internet, such as downloading patches or calling external APIs, without being directly accessible from the internet.
- How it Works: NAT (Network Address Translation) rewrites internal IP addresses to a public IP on outbound traffic. The NAT gateway handles return traffic and blocks inbound connections.
- One-Way Traffic: Only outbound communication is allowed from the private subnet — the internet cannot initiate connections back.
- Use Case: Private VMs needing access to package repositories, external APIs, or software updates without being exposed publicly.

networking vpn



VPN Gateway

- Purpose: Creates a secure, encrypted tunnel between your cloud network and an on-premises environment or another cloud network over the public internet.
- How it Works: A VPN gateway is paired with a customer gateway. Together, they establish a secure connection that allows both networks to communicate as if they were on the same private network.
- Use Case: Hybrid cloud setup — securely extending your on-premises infrastructure into the cloud to support cloud migrations, backups, or shared applications.

Q4 What purpose do Instance-Level and Network-Level Firewalls serve?

Why Firewalls in Cloud?

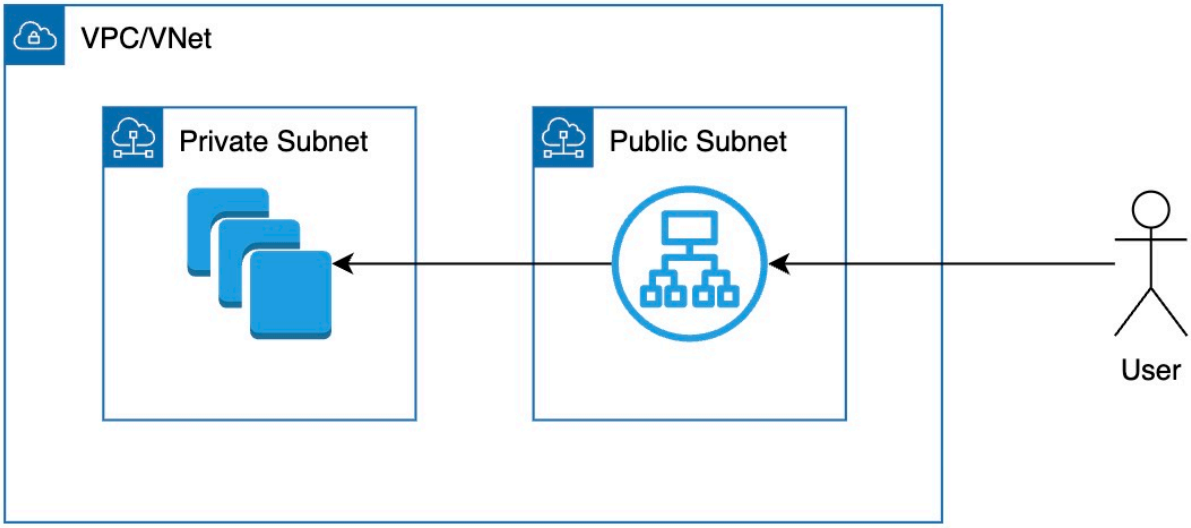
- Scenario: Imagine deploying an application without any protection. Anyone can access any port. Security risks are high.
- Firewalls: Help control who can access your resources and what kind of traffic is allowed.

What is a Firewall?

- Definition: A set of rules that allow or deny network traffic
- Goal: Protect applications by filtering traffic based on IP, port, or protocol

networking vpc subnet

Instance-Level Firewalls



- Attached to a Resource: Applied directly to virtual machines or containers
- Control Inbound & Outbound: Define what traffic can reach or leave an instance
- Example Rules:
  Allow SSH (port 22) only from your office IP
  Allow HTTP (port 80) from anywhere

Network-Level Firewalls

- Applied at Subnet or VPC Level: Control traffic for all resources in a section of the network
- Example Rules:
  Block all incoming traffic by default
  Allow internal traffic between subnets

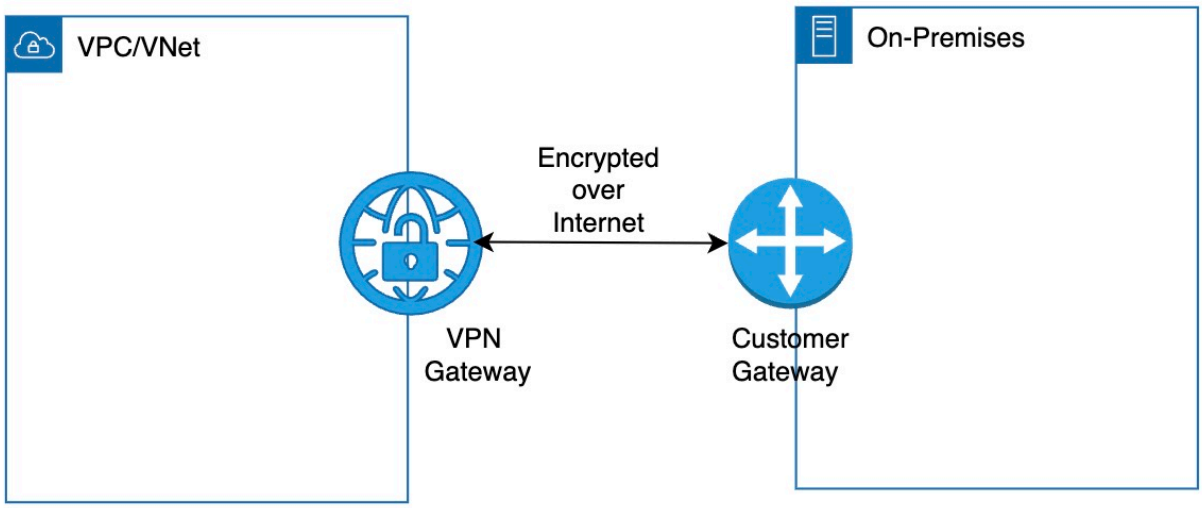
Key Differences

- Instance-Level: Granular control, applied to specific resources
- Network-Level: Broader control, applied to entire segments of your network
- Both Can Work Together: Use layered security for better protection

Q5 What is the need for Hybrid Networks?

Why Hybrid Networks?

- Scenario: Imagine a company that wants to use the cloud for scalability but still runs critical apps on-premises. They need both to talk securely and reliably.
- Hybrid Network: Connects on-premises infrastructure with cloud resources — so both can work together seamlessly.



networking vpn

What is a Hybrid Network?

- Definition: A network setup that securely connects on-premises data centers with cloud environments
- Goal: Extend your private network to the cloud without exposing everything to the internet
- Use Cases:
  - Gradual cloud migration
  - Data residency or compliance requirements
  - Low-latency access to on-prem systems
  - Disaster recovery and backup

Key Options

- VPN (Virtual Private Network):
  - Encrypted connection over the internet
  - Cost-effective but may have limited bandwidth
- Dedicated Private Network (Interconnect/ExpressRoute/Direct Connect):
  - Private, high-bandwidth, low-latency link
  - Best for high-volume data transfer

Q6 Why do you need DNS service in Cloud?

Why DNS in Cloud?

- Scenario: Imagine accessing a website using an IP address like 192.168.1.5. Not easy to remember. What if the IP changes?
- DNS: Converts friendly names (like myapp.com) to IP addresses, so users and systems can connect easily.

What is DNS?

- Definition: Domain Name System – a phonebook for the internet
- Goal: Translate human-friendly names into machine-friendly IPs
- Example: app.mycompany.com 35.220.45.123

How DNS Works

- Step 1: User types a name like myapp.com
- Step 2: DNS finds the matching IP address
- Step 3: Traffic is sent to that IP address

DNS in the Cloud

- Fully Managed: Cloud handles scaling, reliability, and speed
- Globally Distributed: DNS servers are close to users – low latency

Use Cases in Cloud

- Access Services: Map domain names to load balancers, servers, or APIs
- Internal DNS: Name resources inside your cloud network (e.g., db.internal)

Advanced DNS Features

- Routing Policies: Send users to the nearest server (geo-routing)
- Health Checks: Automatically remove unhealthy endpoints
- Failover: Redirect to a backup if primary fails

Q7 Networking Components/Services Across Cloud Platforms

Component	Purpose	AWS	Azure
Virtual Network	Create isolated cloud networks	VPC	Virtual Network (VNet)
Subnets	Divide virtual networks by access level	Subnets	Subnets
CIDR Blocks	Allocate and organize IP ranges	CIDR for VPC and subnets	CIDR for VNets and subnets
Internet Gateway	Allow public internet access for resources with public IPs	Internet Gateway	Internet Gateway (via NSG + LB)
NAT Gateway	Allow private resources to access internet without inbound exposure	NAT Gateway	NAT Gateway
Firewalls – Instance	Apply security rules at individual resource level	Security Groups	NSG (Network Security Group)
Firewalls – Network	Apply security rules at subnet or network level	NACLs	NSG (per subnet)
Hybrid Networking	Connect cloud with on-prem systems	Direct Connect, VPN	ExpressRoute, VPN
DNS Service	Map domain names to cloud services and endpoints	Route 53	Azure DNS
VPC Peering	Connect two virtual networks privately without using gateways	VPC Peering	VNet Peering
Private Link / Endpoints	Privately connect to managed services without using public IPs	PrivateLink	PrivateLink

Network Monitoring & Logs	Track traffic, debug performance, and ensure compliance	VPC Flow Logs	NSG Flow Logs
IPv6 Support	Enable IPv6 addressing in virtual networks	IPv6 in VPC	IPv6 in VNet

CHAPTER 5

Cost Management in the Cloud

6 Questions & Answers

5 Cost Management in the Cloud

Q1 What is Total Cost of Ownership (TCO)?

What is Total Cost of Ownership (TCO)?

- Definition: The complete cost of owning, operating, and maintaining a system over its lifecycle
- Goal: Make informed decisions throughout the cloud lifecycle, from migration to ongoing operations

cost management 1

What Does TCO Include?

- Infrastructure Costs: Servers, databases, storage, routers, maintenance
- Licensing Costs: Software (OS, DB, third-party), hardware support contracts
- Networking Costs: Connectivity, VPNs, data ingress/egress
- Personnel Costs: Developers, testers, operations, security, business teams
- Other Costs:
  - SLA penalties: Costs incurred for failing to meet service level agreements
  - Third-party APIs: External APIs you make use of
  - Electricity and facility overhead: Costs to run your data centers

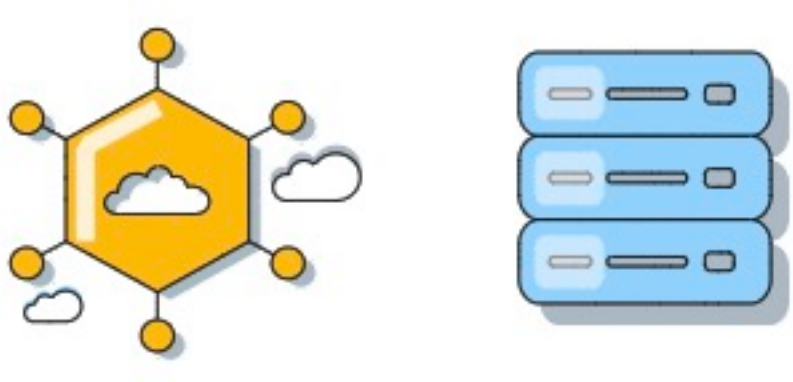
Why TCO Matters?

- Hidden Costs Exist: Data transfer, support, backups, security tooling
- Not Just Infrastructure: Human costs and licensing can be significant
- Compare Fairly: Always compare on-prem vs. cloud based on full TCO — not just hardware

Q2 Compare CapEx vs OpEx

What is Capital Expenditure (CapEx)?

- Definition: One-time investment to purchase infrastructure or licenses
- Examples:
  - Building your own data center
  - Purchasing reserved instances (Example: 3-year commitment)
  - Buying or leasing software licenses
  - BYOL (Bring Your Own License) model



cost management 2

What is Operational Expenditure (OpEx)?

- Definition: Ongoing expense to use services without owning them
- Pay-as-You-Go: Pay based on usage — no upfront investment
- Examples:
  - Running cloud VMs or managed databases
  - Using Serverless functions (FaaS) and paying per invocation
  - Monthly subscriptions to SaaS tools

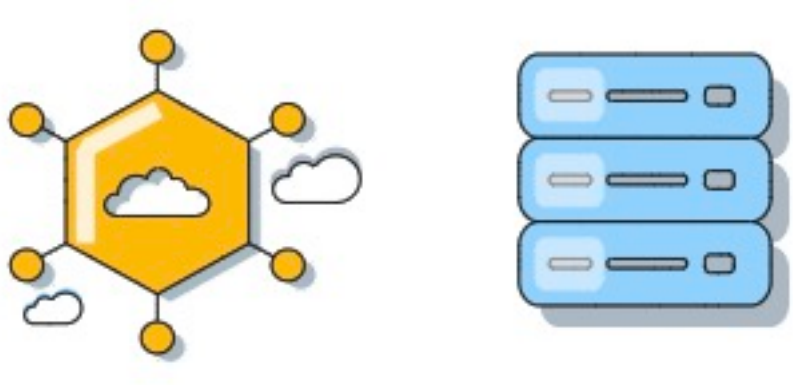
CapEx vs OpEx

Aspect	**CapEx (Capital Expenditure)**
Purpose	Long-term investment in assets
Payment Timing	Paid upfront
Examples	Buying servers, data centers, network gear, long-term reservations
Flexibility	Low — fixed once purchased

Q3 Compare Consumption-Based Pricing vs Fixed-Price Pricing

What is Consumption-Based Pricing?

- Definition: You only pay for what you use — nothing more
- Highly Efficient: Great for workloads with unpredictable usage
- Examples:
  - Serverless Functions (FaaS) — pay per invocation
  - API Gateway — pay per request
  - Cloud Storage — pay per GB stored and transferred



cost management 2

What is Fixed-Price Pricing?

- Definition: You pay a fixed amount whether or not the resource is fully used
- Predictable Billing: Good for steady workloads
- Can Waste Resources: You are billed even if the system sits idle
- Examples:
  - Provisioned VM — billed for uptime regardless of usage
  - Kubernetes Cluster — billed for reserved nodes even if underutilized

---

Q4 Best Practices for Managing Cloud Costs

Why Manage Cloud Costs Proactively?

- What Can Happen?: Cloud makes it easy to launch resources, but without controls, bills can grow fast.
- Cost Management: Helps you stay in control, avoid surprises, and align spending with business goals.

cost management 3

Best Practices for Managing Cloud Costs

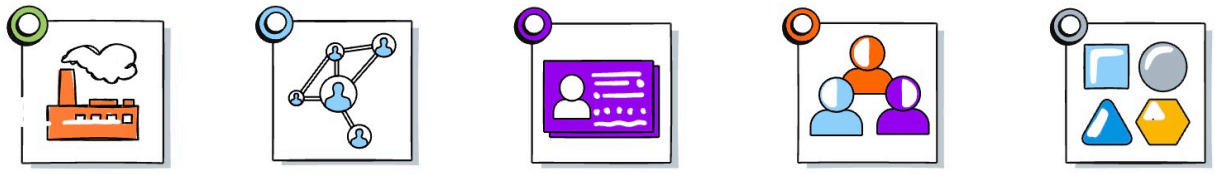
- Estimate Costs Before You Deploy
  - Use pricing calculators to forecast cloud spend
  - Plan for scaling needs
- Review Costs Regularly
  - Weekly or bi-weekly reviews to catch anomalies early
  - Shift from CapEx planning to OpEx monitoring
  - Involve all stakeholders — tech, finance, business, and exec teams
- Use Cost Management Tools
  - Set up budgets and alerts to stay within limits
  - Analyze usage trends to make informed decisions
- Group Resources by Ownership
  - Use tags or labels
  - Helps assign costs to teams, projects, or environments
- Optimize Resource Usage
  - Stop or delete idle resources (if possible automate)
  - Use managed services (PaaS over IaaS) for better efficiency (Cloud provider handles underlying infrastructure, scaling, and maintenance, often leading to lower operational overhead)
  - Use spot/preemptible instances for non-critical, fault-tolerant workloads
  - Reserve compute resources for 1 or 3 years for steady workloads

Q5 What is FinOps?

Why FinOps?

- What Can Happen?: Imagine your cloud bill doubling overnight and no one knows why. Teams move fast, but without cost awareness.
- FinOps: A cultural practice that brings together engineering, finance, and business teams to manage cloud costs effectively.

cost management 1



What is FinOps?

- Definition: Cloud Financial Operations (FinOps) is a framework to track, optimize, and forecast cloud spending
- Goal: Achieve cost efficiency without slowing down innovation
- Philosophy: Everyone takes ownership of cloud usage and cost

Core Principles of FinOps

- Teams collaborate on cloud cost management
- Everyone is accountable for their cloud usage
- Decisions are driven by business value
- Centralized visibility, but decentralized responsibility

Key Activities in FinOps

- Visibility:
  - Tag resources by team, project, or environment
  - Use dashboards to track usage and cost by service or app
- Optimization:
  - Rightsize compute resources (e.g., downscale over provisioned VMs)
  - Turn off idle resources (e.g., unused dev/test environments)
  - Use the right mix of on-demand instances, savings plans, reserved instances, and/or spot VMs
- Budgeting & Forecasting:
  - Set budgets per team/project
  - Predict future costs using past usage patterns
- Automation:
  - Use policies to auto-delete unused resources
  - Alert teams on anomalies or budget threshold breaches

FinOps Lifecycle

- Inform – Understand and share current costs and usage
- Optimize – Identify ways to reduce spend and increase efficiency
- Operate – Continuously monitor, report, and improve cloud financial practices

Q6 What are the Key Cost Management Tools in Different Cloud Platforms?

Category	Explanation	AWS	Azure
Cost Visibility	Visualize and analyze current and historical cloud spend	AWS Cost Explorer	Cost Management + Billing
Budgets & Alerts	Set budget limits and trigger alerts on overspend	AWS Budgets	Budget alerts
Pricing Estimation	Estimate service costs before provisioning	AWS Pricing Calculator	Azure Pricing Calculator
Tagging & Grouping	Categorize resources for tracking and chargeback	AWS Resource Tags	Tags
Savings Plans & Discounts	Commit usage for long-term savings	Reserved Instances, Savings Plans	Reserved Instances, Savings Plans
Idle Resource Detection	Identify unused or underused resources	AWS Compute Optimizer	Azure Advisor
Cost Anomaly Detection	Detect unexpected spikes in spending	Cost Anomaly Detection	Azure Cost Management + Cost Analysis Insights

CHAPTER 6

AI, ML and Generative AI in the Cloud

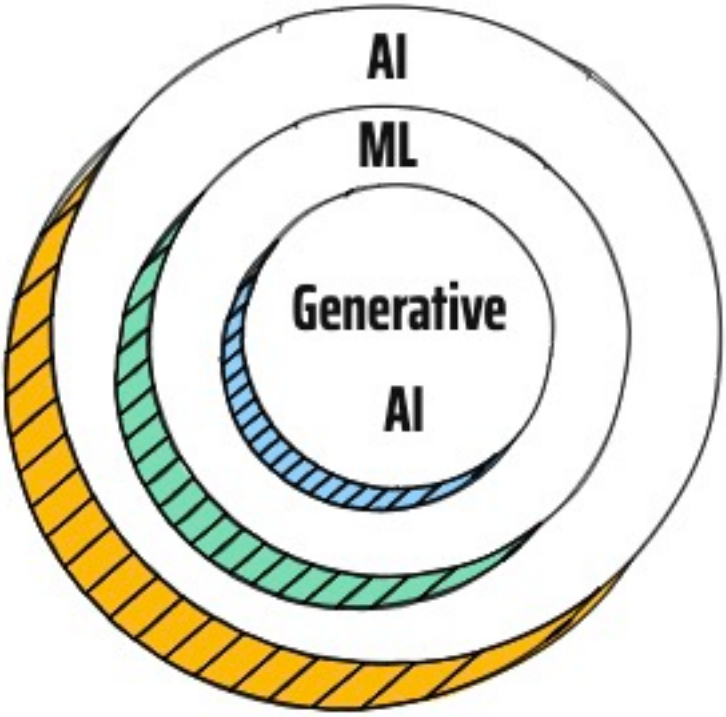
5 Questions & Answers

6 AI, ML and Generative AI in the Cloud

Q1 Compare AI vs ML

What is Artificial Intelligence?

- Definition: A branch of computer science focused on building systems that can simulate human intelligence
- Goal: Enable machines to see, understand, learn, and make decisions



ai ml gen ai

AI is All Around Us

- Self-driving Cars: Sense environment and make driving decisions
- Spam Filters: Detect and block unwanted emails
- Fraud Detection: Analyze behavior to flag suspicious transactions
- Recommendation Systems: Suggest music, movies, products based on past activity
- Email Sorting: Categorize emails into Primary, Promotions, and Social tabs

How is Traditional Programming done?

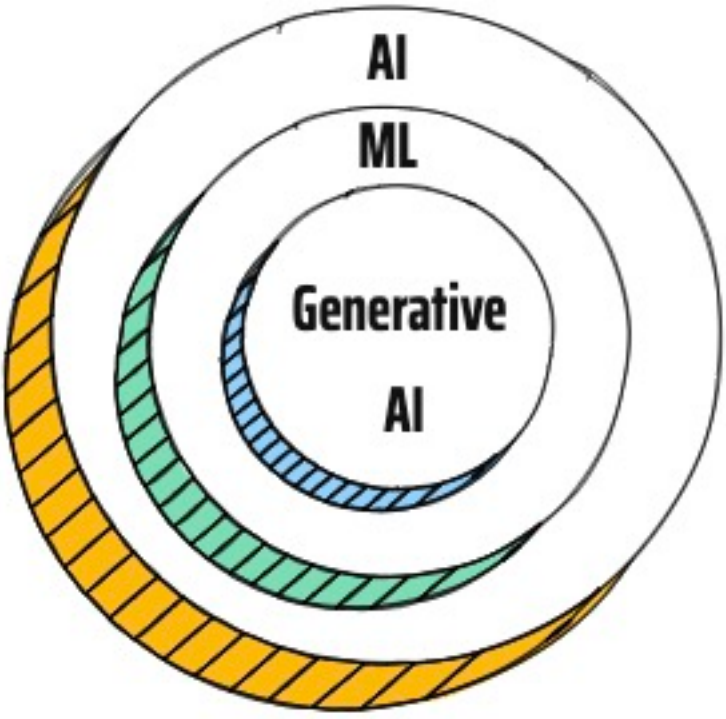
- Scenario: You want to predict house prices
- Traditional Programming: You write explicit rules — “IF location is X AND size is Y THEN price is Z”

How does Machine Learning work?

- What Happens?: ML system that learns from data instead of rules
- Machine Learning: Give examples ML finds patterns Predicts outcome
- Power: ML improves as more data is provided

How Machine Learning Works

- Step 1: Feed Data – Provide historical examples
- Step 2: Train Model – ML learns patterns in data
- Step 3: Predict – Use trained model to predict on new data



ai ml gen ai

Aspect	Artificial Intelligence (AI)
Purpose	Simulate human intelligence
Scope	Broad — includes reasoning, learning, and problem-solving
Relation	ML is a subset of AI
Data Dependency	May or may not use data

Q2 What are the Key Steps in Machine Learning Lifecycle?

ai ml stages

Step 1: Data Ingestion

- Collect data from real-world sources
- Examples: User clicks, sales records, logs, images, transaction systems, big data systems, ..

Step 2: Data Preparation

- Clean and transform data
- Examples: Remove duplicates, handle missing values, ..

Step 3: Model Training

- Use training data to build a prediction model
- Use frameworks like TensorFlow, PyTorch, scikit-learn

Step 4: Model Deployment

- Make model available to real-world systems
- Examples: Integrate model into mobile app or website

Step 5: Model Management

- Track model performance over time
- Actions: Retrain with new data

End-to-End Example: Spam Detection

- Ingest: Pull email history
- Prepare: Label spam vs not spam
- Train: Learn patterns from labeled data
- Deploy: Filter incoming emails
- Manage: Keep accuracy high as spam tactics evolve

Q3 What is Generative AI?

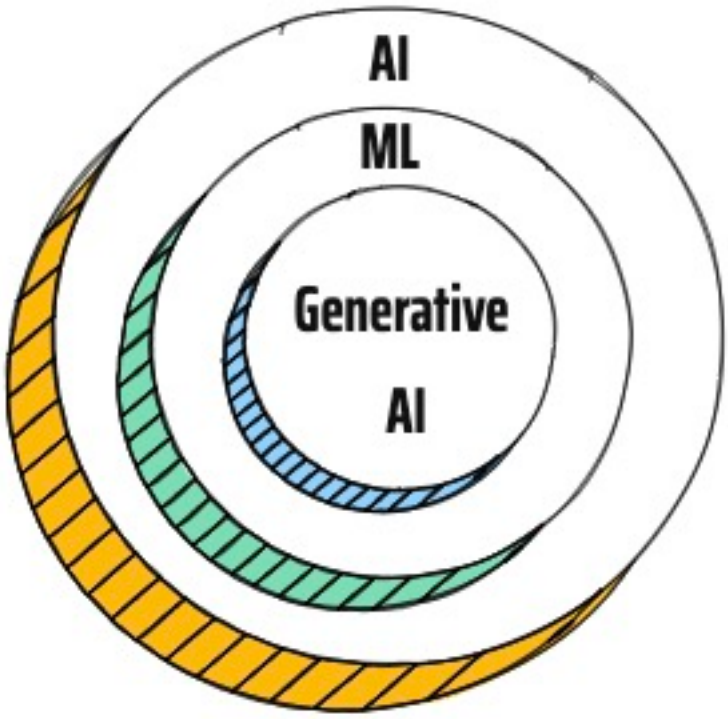
Why Generative AI?

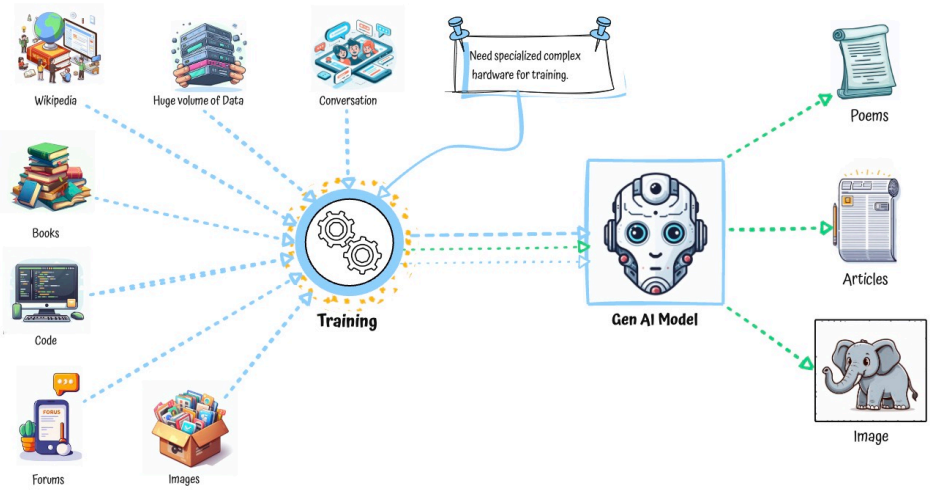
- Scenario: Traditional ML models can detect spam or predict house prices. But what if we want the machine to write an email, generate an image, or create a video?
- Generative AI: A special kind of AI that creates new content like text, code, images, and audio — not just make predictions.
- Definition: A subset of Machine Learning that creates original content from patterns it learned
- Goal: Move from consuming data to generating data
- Examples: Writing poems, answering questions, generating software code, creating art

ai ml gen ai

How is Generative AI Different from Traditional ML?

Aspect	Generative AI
Purpose	Create new content (text, images, code, etc.)
Output Type	Original and creative outputs
Examples	ChatGPT, DALL·E, GitHub Copilot
Training Data	Requires massive datasets and compute power
Use Cases	Content generation, summarization, coding help





ai ml gen ai complex training

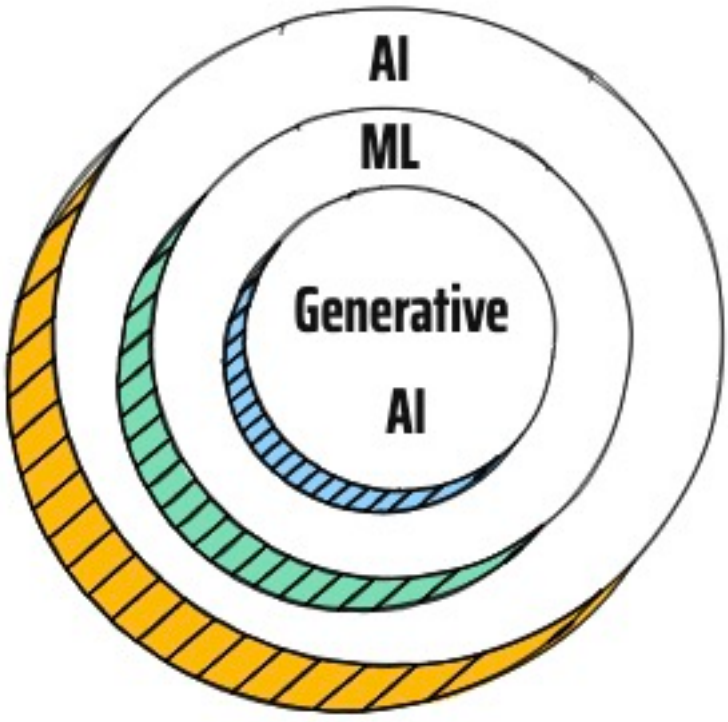
> How does Generative AI Work?

- Foundation Models Power Generative AI
  - Definition: Huge pre-trained models trained on broad data (text, code, images)
  - Reusable: One model can handle many tasks (chat, translate, summarize)
  - Customizable: Many apps fine-tune foundation models for their specific needs
- Training Foundation Models is Complex
  - Massive Datasets: Learn from books, code, web pages, image captions
  - Expensive Training: Needs GPUs, TPUs, distributed storage
  - Takes Time: Training can take weeks or months, depending on size
- Key Limitations of Foundation Models
  - Data Dependency: Garbage in, garbage out
  - Model quality depends on training data
  - If trained on biased or outdated info, it will produce flawed results
  - Knowledge Cutoff: Models freeze in time
  - Cannot access real-time data
  - Cannot tell you what happened after their training period
  - Hallucinations: Confident but incorrect responses
  - May generate fake facts, URLs, citations, or statistics
  - Bias: Inherits bias from training data
  - Can unintentionally favor certain groups over others
  - Edge Cases: Uncommon inputs often fail
  - Models may behave unexpectedly with unusual or rare queries
- Reducing Hallucinations with Grounding
  - Grounding: Link AI outputs to verified data
  - Example: Connect chatbot to trusted company documents
  - Benefits:
  - Reduces hallucinations
  - Provides sources
  - Popular Technique – RAG (Retrieval-Augmented Generation)
  - Step 1: Search trusted documents
  - Step 2: Add findings to the prompt
  - Step 3: Let model generate based on that data

Q4 How did the ML landscape change after Generative AI?

ML Landscape Before Generative AI

- 3 Typical Approaches: Prebuilt ML APIs, AutoML tools, or full custom models
- ML APIs: Use ready-made APIs for text, speech, image, or video tasks
  - Use Case: Convert customer calls to text using Speech-to-Text API
  - Use Case: Detect objects in product images using Vision API
  - Use Case: Classify customer reviews as positive or negative
- AutoML: Train custom models without ML expertise by uploading labeled data — platform handles training and deployment
  - Use Case: Classify support tickets into categories using labeled examples
  - Use Case: Categorize pet images (e.g., dogs vs cats) by uploading a labeled dataset
- Custom Models: Full control for ML teams using frameworks like TensorFlow, PyTorch, scikit-learn — bring your own data and code
  - Use Case: Build a custom model to forecast daily sales using historical data
  - Use Case: Train a deep learning model for detecting anomalies in financial transactions



ai ml gen ai

> Generative AI Brought in New Capabilities

- Foundation Models Introduced: Pre-trained on massive datasets to generate text, code, audio, and images
    - Use Case: Generate product descriptions automatically for e-commerce listings
    - Use Case: Write code snippets based on natural language prompts
- New Capabilities: Summarization, chat, translation, and content generation via simple APIs
    - Use Case: Summarize long customer feedback into short highlights
    - Use Case: Translate product manuals into multiple languages
- Models Can Be Tuned Easily: Foundation models can be fine-tuned with your own data to adjust tone, context, or behavior
    - Use Case: Fine-tune a chatbot to use your company's tone and answer product-specific questions

ML Landscape After Generative AI

- Use Pretrained APIs (NO CHANGE) — Use ready-made APIs: text, speech, image, or video tasks
- Use Foundation Models via API (NEW) — New capabilities: chat, summarize, translate, generate code
    - Use Case: Summarize long customer feedback into short highlights
- Customize Foundation Models (NEW) — fine-tune on your own data
    - Use Case: Fine-tune a chatbot to use your company's tone and answer product-specific questions
- Train with AutoML (NO CHANGE) — build models without writing code
    - Use Case: Categorize pet images (e.g., dogs vs cats) by uploading a labeled dataset
- Build Custom Models (NO CHANGE) — use your own infra and tools if full control is needed
    - Use Case: Train a deep learning model for detecting anomalies in financial transactions

Key Shift After GenAI

- Platforms now support both traditional ML and GenAI paths

Q5 Managed Services In Different Cloud Platforms for AI and ML

Category	Description	AWS	Azure
Use Pretrained APIs	Use ready-made APIs: text, speech, image, or video tasks	Amazon Rekognition (image & video analysis) Amazon Comprehend (text analytics),..	Azure AI Vision (image analysis) Azure AI Language (text analysis), ..
Use Foundation Models via API	New capabilities: chat, summarize, translate, generate code	Amazon Bedrock	Azure AI Foundry
Customize Foundation Models	Fine-tune foundation models on your own data (cloud-specific & open-source models such as Anthropic Claude, Meta Llama, & Mistral)	Amazon Bedrock	Azure AI Foundry
Train with AutoML	Build models without writing code	Amazon SageMaker Autopilot	Azure Machine Learning Automated ML
Build Custom Models	Platforms for building, training, and deploying custom ML models using TensorFlow,...	Amazon SageMaker	Azure Machine Learning

CHAPTER 7

Security in the Cloud

7 Questions & Answers

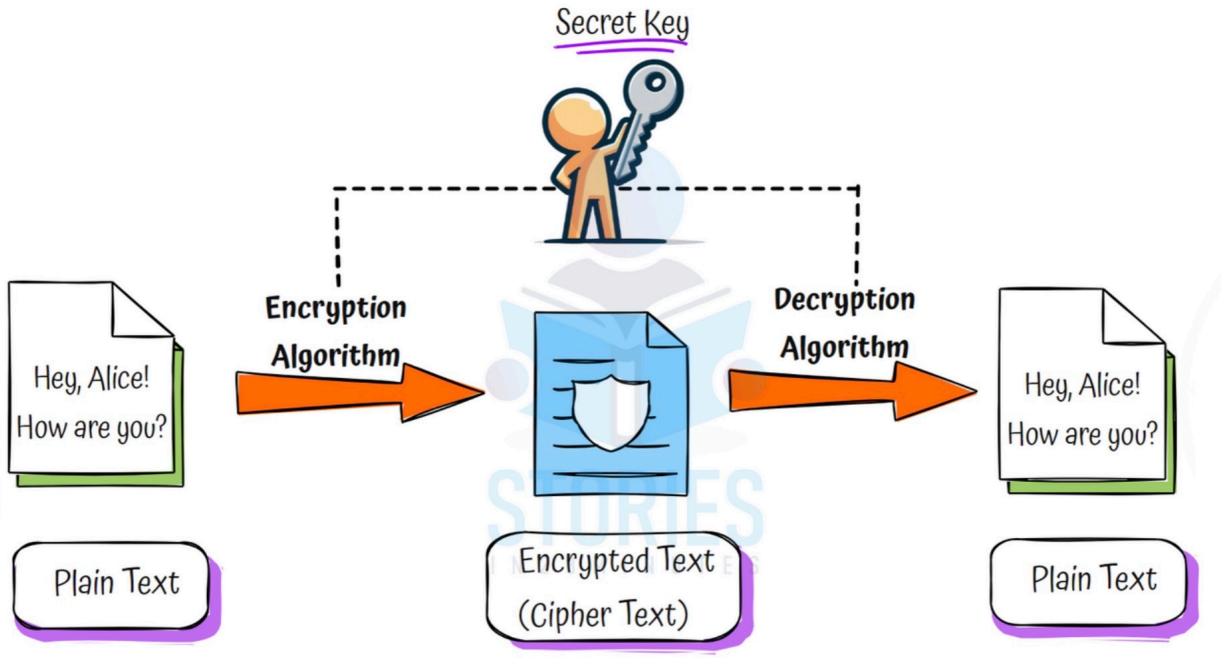
7 Security in the Cloud

Q1 What is Key Management Service?

Why Encryption?

- Scenario: You are storing credit card details or personal data in a system. What if someone hacks the database? We need a way to protect sensitive information even if it gets stolen.
- Encryption: Converts data into unreadable format using keys – only someone with the right key can read it.

Symmetric Key Encryption



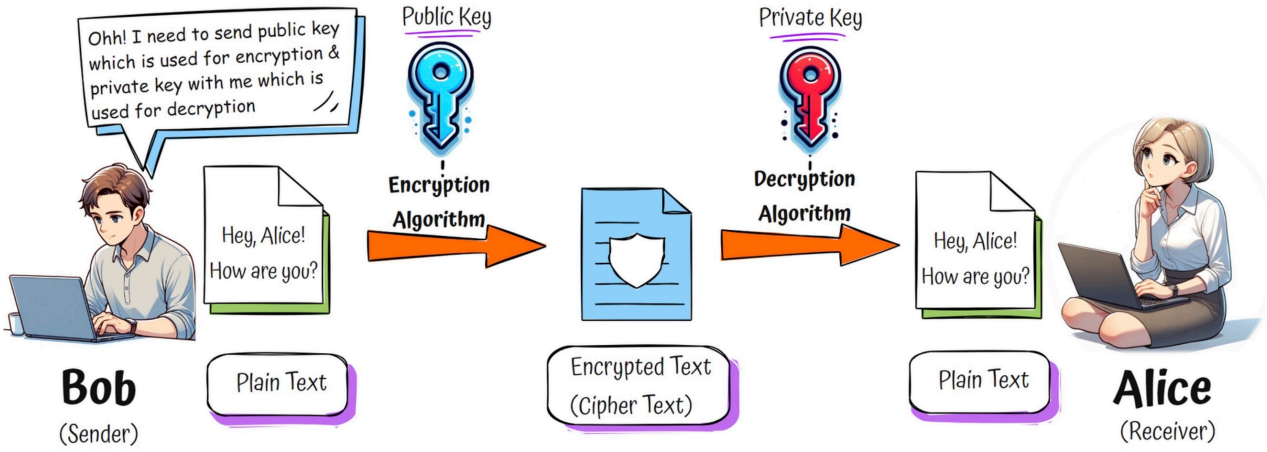
encryption one key

- One Key: Same key is used to encrypt and decrypt data
- Simple & Fast: Very efficient for encrypting large amounts of data
- Problem: Key must be shared with others securely – that's risky!

Key Questions for Symmetric Encryption

- Which Algorithm?: Choose safe options like AES
- How to Protect Key?: Store securely (e.g., in Key Management Services)
- How to Share Key?: Avoid sending it directly – consider using asymmetric encryption to share it!

Asymmetric Key Encryption (Public Key Cryptography)



encryption two keys

- Two Keys:
  - Public Key: Shared with everyone
  - Private Key: Kept secret
- Encryption Flow:
  - Encrypt with Public Key
  - Decrypt with Private Key
- Use Case: You can receive data securely from anyone without needing to share a key

Common Misconception

- Public Key Easy to Reverse: Math behind it (e.g., RSA) makes it nearly impossible to compute the private key from the public key

What is Key Management Service?

- Definition: A service that helps create, store, rotate, and manage encryption keys
- Goal: Keep your data secure — at rest and in transit — using strong encryption practices

Services That Typically Use Key Management Services

- Object Storage – Encrypt files and backups
- Block Storage – Protect disk volumes for VMs and databases
- Databases – Encrypt data at rest
- Messaging Services – Secure messages in queues or topics
- Secrets Management – Store and encrypt passwords, tokens, and API keys
- Backup & Disaster Recovery – Encrypt snapshots and backup data
- Logging & Monitoring – Encrypt logs stored in logging services

HSM (Hardware Security Module)

- Scenario: Imagine storing ultra-sensitive data like government records, banking transactions, or healthcare info. You want the encryption keys to be super secure — even cloud provider admins shouldn't access them.
- What is HSM?: A physical device (available in the cloud too) that stores and manages encryption keys with strong protection.
- Highest Level of Security: HSMs offer FIPS 140-2 Level 3 compliance — trusted by banks, governments, and regulated industries.
- Dedicated and Isolated: In the cloud, HSMs are not shared. Only your organization has access to your HSM instance.
- Use Case: Needed when regulations require physical separation and dedicated control of key material.
- Cloud Examples: AWS CloudHSM, Azure Dedicated HSM, Google Cloud HSM.

Cloud Managed Services for Key Management and HSM

- AWS: AWS Key Management Service (KMS), AWS CloudHSM
- Azure: Azure Key Vault, Azure Dedicated HSM
- Google Cloud: Google Cloud Key Management Service (KMS), Google Cloud HSM

Q2 Encryption at Rest vs Encryption In Transit

What is Encryption at Rest?

- Definition: Encrypts data when it's stored on disk – like in databases, storage, or backups
- Goal: Prevent unauthorized access if storage is compromised
- Use Cases: Object storage, block storage, database files, snapshots
- Managed By: Server-side encryption using KMS or HSM (in most cases)

What is Encryption in Transit?

- Definition: Encrypts data while it's being transmitted across networks
- Goal: Protect data from being read or modified while moving between users, apps, or services
- Use Cases: API calls, browser access, microservices
- How is it done?: Uses secure protocols like HTTPS and mTLS, powered by SSL/TLS certificates

HTTPS: Commonly used for user-to-application communication — for example, when a user accesses a web app through a browser or an app calls an external API

mTLS (Mutual TLS): Used for service-to-service communication within secure environments like microservices or internal APIs — both client and server authenticate each other using certificates for stronger trust and protection

1: What is the Role of SSL/TLS Certificates?

- Identity Verification: Confirms that the server you are connecting to is the real example.com — not an imposter
- Encryption Enablement: Helps establish a secure, encrypted connection between client and server using HTTPS
- Trust Establishment: Issued by trusted Certificate Authorities (CAs), making browsers trust the website
- Foundation of HTTPS: Without a valid SSL/TLS certificate, HTTPS cannot be established — the browser will warn users
- Used in Handshake: The certificate is shared during the handshake to initiate secure communication and exchange encryption keys securely

SSL vs TLS

- SSL (Secure Sockets Layer): The original encryption protocol for securing web traffic — now outdated and insecure
- TLS (Transport Layer Security): The modern, more secure version that replaced SSL
- Current Standard: TLS 1.2 and TLS 1.3 are widely used today; SSL is deprecated
- Naming Confusion: People still say “SSL certificate,” but TLS is what’s actually used
- Bottom Line: When you see HTTPS, it's powered by TLS, not SSL — even if the certificate is called an “SSL certificate”

2: How does HTTPS work?

- Step 01: A user opens a website by typing https://example.com in the browser
- Step 02: The browser contacts the server and requests a secure connection — this begins the TLS handshake
- Step 03: The server responds with its digital certificate (SSL/TLS certificate) issued by a trusted Certificate Authority (CA)
- Step 04: The browser verifies the certificate to confirm the server's identity is valid and trusted
- Step 05: The browser and server negotiate encryption parameters and securely establish a session key using asymmetric encryption
- Step 06: Both the browser and server now use this session key to encrypt and decrypt all further communication
- Widely Used: To access websites securely through a browser

3: How does Mutual TLS(mTLS) work?

- Step 01: A client (typically another app) tries to connect to a secure server using HTTPS
- Step 02: The server sends its TLS certificate to the client to prove its identity
- Step 03: The client verifies the server's certificate using a trusted Certificate Authority (CA)
- Step 04: Unlike standard HTTPS, the server now asks the client for its own certificate
- Step 05: The client sends its client certificate to the server
- Step 06: The server verifies the client's certificate to confirm the client's identity
- Step 07: Once both sides are authenticated, they generate a shared session key for encrypted communication
- Step 08: All future data is encrypted and decrypted using the session key — both client and server are trusted
- Widely Used: In secure microservice communication

Q3 What are DDoS Attacks?

What is a DDoS Attack?

- Scenario: Your website or service suddenly becomes slow or unavailable. It is not a bug — it is being overwhelmed by fake traffic.
- Goal of the Attacker: Make your service unusable by flooding it with more traffic than it can handle.

DDoS = Distributed Denial of Service

- Definition: A cyber attack where multiple machines send massive traffic to a server, app, or network
- Goal: Deny access to legitimate users by exhausting system resources

Impact of DDoS Attacks

- Downtime: Website or app becomes unreachable
- Revenue Loss: E-commerce and service platforms lose sales
- Reputation Damage: Users may lose trust
- Resource Waste: Servers and bandwidth consumed by fake traffic

DDoS Protection in the Cloud

1: Built-In Basic Protection in Many Cloud Services

Service Type	Basic DDoS Protection Built-In?
Cloud Load Balancers	Yes
DNS Services	Yes
Content Delivery Networks (CDNs)	Yes

2:Advanced DDoS Protection

- For mission-critical or high-risk workloads
- Includes:
  - Real-time traffic monitoring
  - 24/7 response support
- Examples:
  - AWS: AWS Shield (Advanced)
  - Azure: Azure DDoS Protection (Standard)
  - Google Cloud: Cloud Armor (with Adaptive Protection)

Q4 What are CVEs?

What are CVEs?

- Scenario: Imagine your app uses an open-source library, and suddenly a security team warns, "It has a CVE!" What does that mean?
- CVE: Stands for Common Vulnerabilities and Exposures — a system for identifying and cataloging publicly known security flaws.
- Definition: A unique ID assigned to a known security vulnerability
- Example: CVE-2024-12345 might refer to a serious flaw in a library your app uses
- Managed by: The MITRE Corporation, in coordination with vendors and researchers
- Goal: Make it easy to communicate, track, and remediate vulnerabilities across systems

Why CVEs Matter in Cloud and DevOps

- Transparency: One global ID for each vulnerability — no confusion
- Automation: Security scanners use CVEs to detect flaws in your code or dependencies
- Patch Management: Helps you quickly know what needs to be updated
- Risk Awareness: CVE severity (via CVSS score) helps prioritize what to fix first

Example Flow

- A vulnerability is found in log4j
- It’s published as CVE-2021-44228
- Security tools scan your cloud resources and alert you
- You patch or mitigate the vulnerability

Cloud Managed Services That Help Track CVEs

- AWS: AWS Inspector (automatic CVE scanning on EC2, Lambda, containers)
- Azure: Microsoft Defender for Cloud (vulnerability assessment)
- Google Cloud: Container Analysis (detects CVEs in images and VMs)

Q5 Where Does OWASP Fit in Cloud Application Security?

Where Does OWASP Fit in Cloud Application Security?

- Scenario: You're building a web application and wondering which threats to prioritize. OWASP helps you focus on the most common and critical ones.
- OWASP: The Open Worldwide Application Security Project—an industry-recognized authority that publishes security best practices, especially the OWASP Top 10.
- Definition: OWASP provides a list of the most critical security risks for web applications
- Goal: Help teams build secure applications by addressing the most likely and most impactful vulnerabilities

Example OWASP 10 List (Changes with Time)

- Broken Access Control Misconfigured IAM roles or API permissions
- Cryptographic Failures Missing encryption at rest or in transit
- Injection SQL/NoSQL injection through unvalidated user input
- Insecure Design Missing security controls like throttling, logging, and input validation filters
- Security Misconfiguration Public S3 buckets, open firewalls, exposed ports
- Vulnerable and Outdated Components Old libraries in cloud-deployed containers
- Identification and Authentication Failures Missing MFA or improper session handling
- Software and Data Integrity Failures No verification of code
- Security Logging and Monitoring Failures Logs not enabled or not monitored
- Server-Side Request Forgery (SSRF) Attacker tricks the server into making requests to internal services (e.g., accessing internal IPs)

Q6 Why Application Firewall in Cloud?

Why Application Firewall in Cloud?

- Scenario: Imagine your app is deployed in the cloud and starts receiving suspicious traffic — SQL injections, bots, DDoS attacks. You need a shield.
- Application Firewall: A protective layer that filters and monitors HTTP/S traffic to your cloud application.

What is a Cloud Application Firewall?

- Definition: A Web Application Firewall (WAF) that protects cloud apps by inspecting incoming requests and blocking malicious patterns
- Goal: Prevent common attacks (like those in OWASP Top 10) before they reach your app
- Deployment: Typically placed in front of APIs, web apps, and load balancers

Key Capabilities

- Threat Detection: Blocks SQL injection, cross-site scripting (XSS), CSRF, SSRF
- Traffic Filtering: Allow or deny requests based on IPs, headers, geolocation
- Rate Limiting: Control traffic spikes
- Bot Protection: Detect and block malicious bots or scrapers
- Logging & Alerting: Monitor threats and alert teams in real time

Best Practices

- Enable default OWASP protection rules
- Set up geo-blocking if your app is regional
- Use rate limiting for login and API endpoints
- Regularly review WAF logs and update rules
- Combine with CDN and DDoS protection for stronger defense

Cloud Managed Services for Application Firewall

- AWS: AWS WAF
- Azure:Azure Web Application Firewall
- Google Cloud: Cloud Armor

Q7 Why Shift Left Security?

Why Shift Left Security?

- Scenario: Imagine discovering a security flaw just before production. Fixing it now delays release, increases cost, and risks customer trust.
- Shift Left Security: Catch and fix security issues early in the development process, rather than waiting until the end.

What is Shift Left Security?

- Definition: Practice of integrating security early in the software development lifecycle (SDLC)
- Goal: Detect vulnerabilities sooner, reduce cost and risk, and deliver secure code faster
- Mindset: Security is everyone's responsibility, not just the security team's job

Key Principles of Shift Left Security

- Early and Continuous: Security checks start during coding, not after deployment
- Integrated Tools: Use security tools inside CI/CD pipelines
- Developer Empowerment: Give developers tools and training to fix issues themselves
- Automation: Automate scans to avoid manual, late-stage bottlenecks

Common Shift Left Security Practices

- Static Application Security Testing (SAST)
  Analyze code for vulnerabilities during development
- Software Composition Analysis (SCA)
  Detect known CVEs in open-source dependencies
- Secrets Scanning
  Prevent hardcoded API keys, passwords, and tokens
- Container and IaC Scanning
  Secure Dockerfiles, Kubernetes configs, Terraform scripts
- Security Unit Tests
  Add tests for auth, input validation, and access control

Benefits of Shifting Security Left

- Faster Fixes: Easier to fix issues while code is still fresh
- Lower Costs: Cheaper to fix bugs early
- Improved Culture: Developers take ownership of security
- Fewer Late Surprises: Fewer last-minute release delays

Cloud Provider	Service
AWS	Amazon Inspector
	Amazon ECR Image Scanning
Azure	Microsoft Defender for Cloud
	Microsoft Defender for Containers
Google Cloud	Artifact Analysis
	OS Configuration (OS Config)

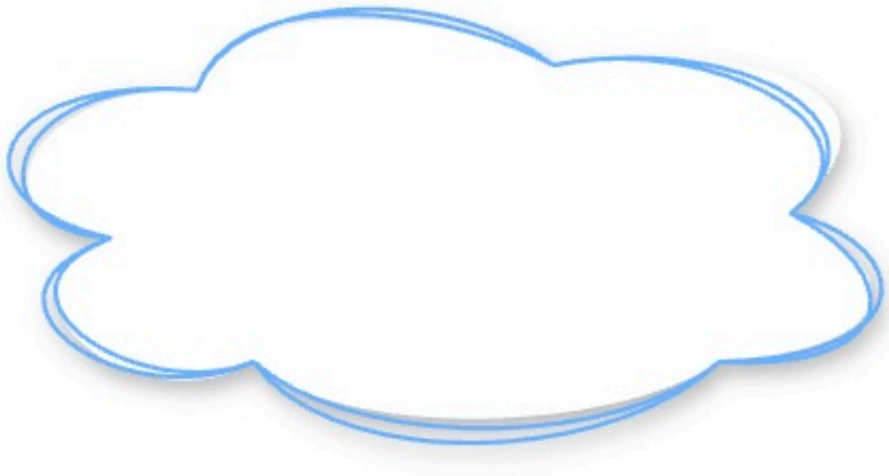
CHAPTER 8

Cloud Interview - Diagrams

10 Questions & Answers

8 Cloud Interview - Diagrams

Q1 Cloud

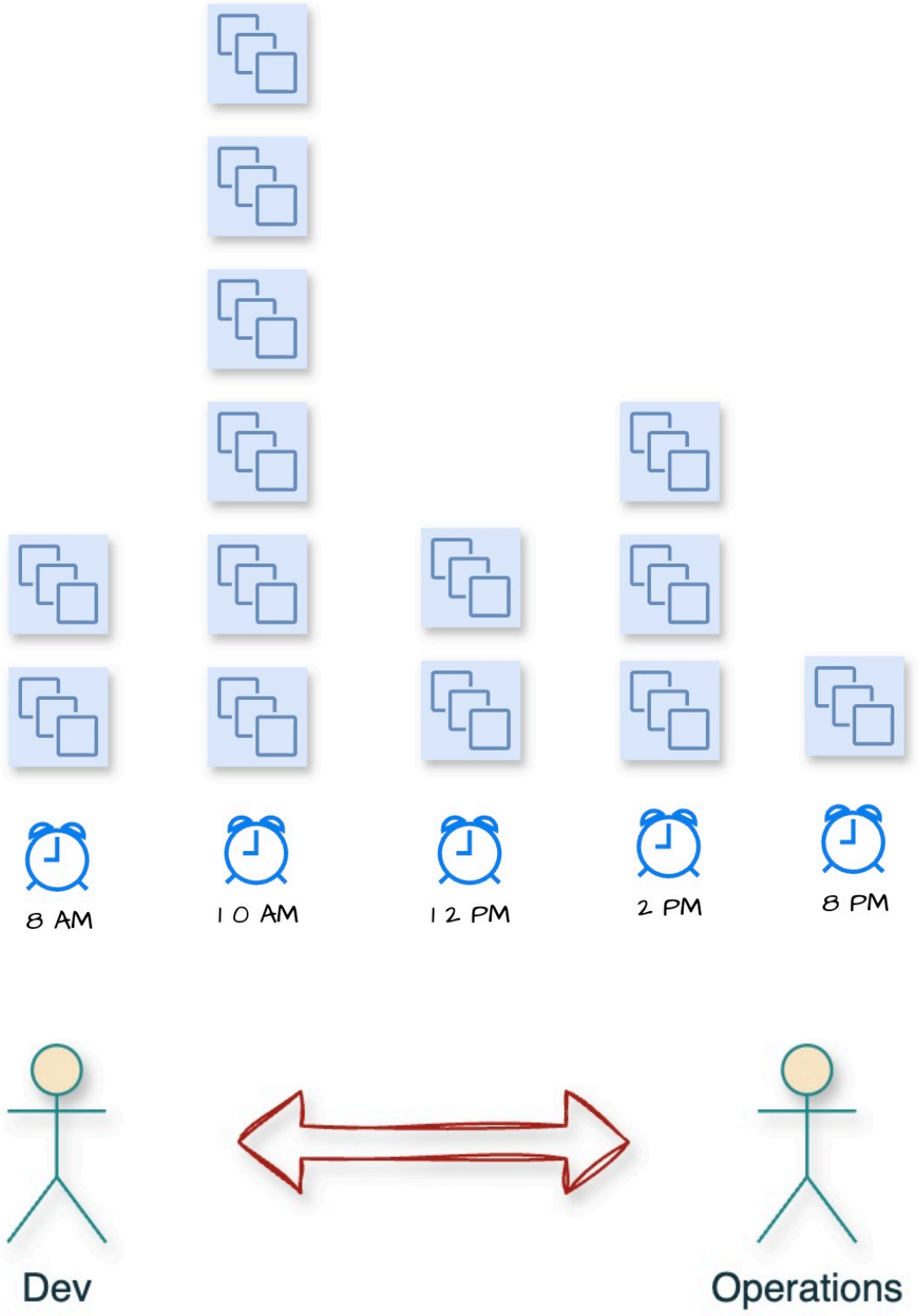


Cloud

Q2 Elasticity

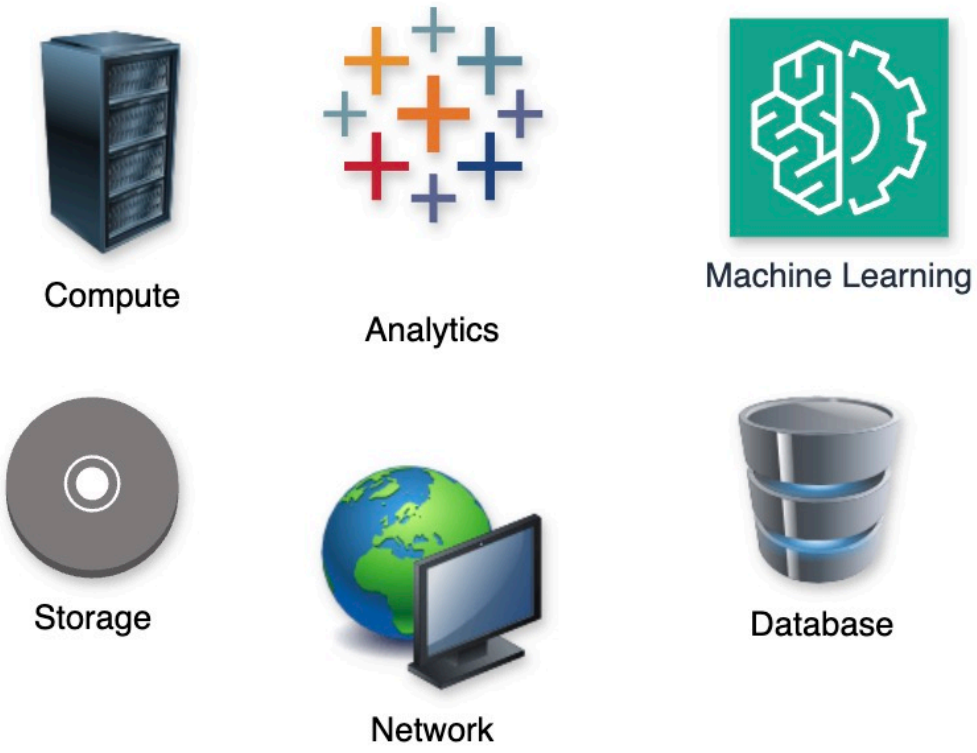
Elasticity

Q3 DevOps



DevOps

Q4 Cloud Services

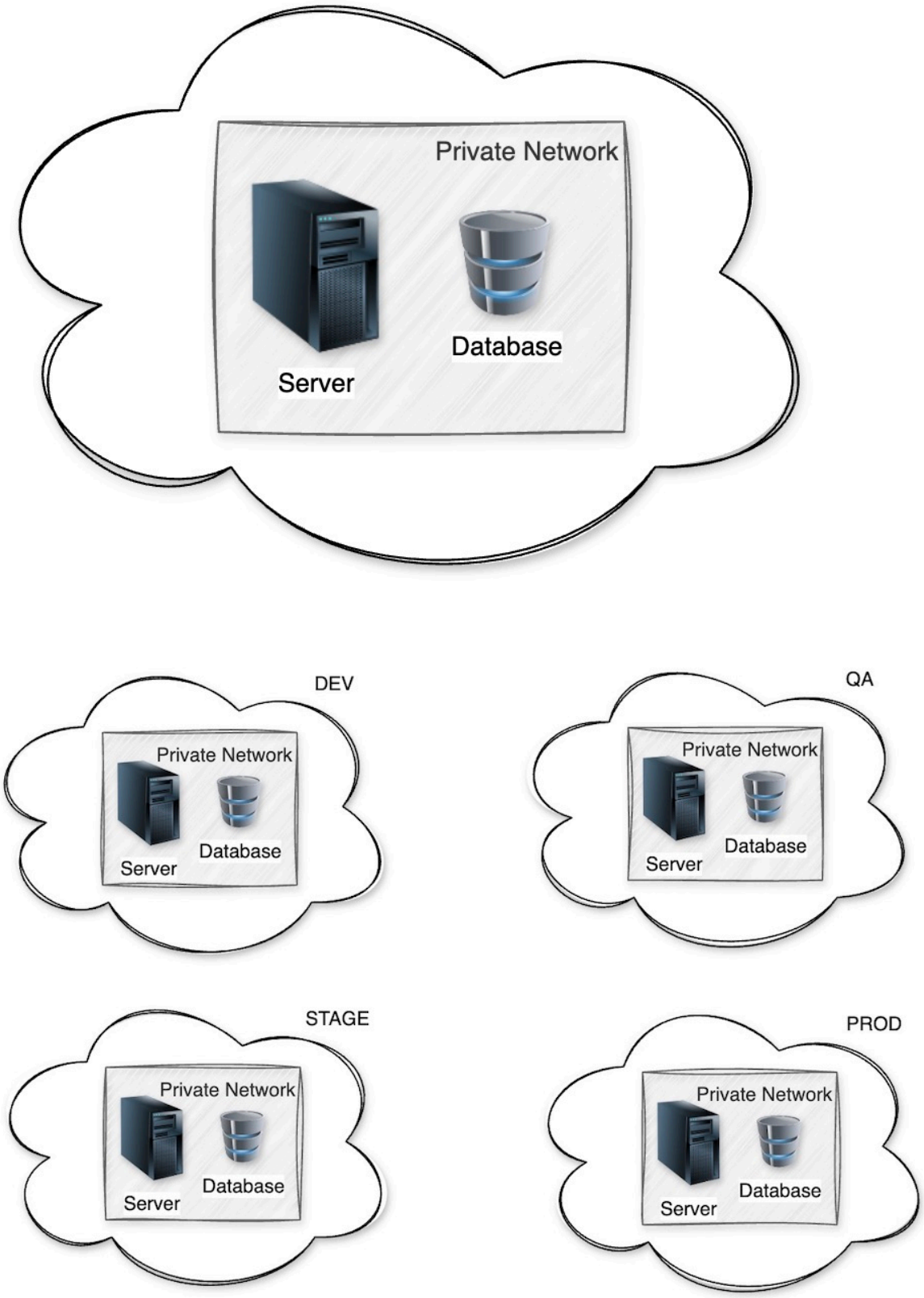


cloud services

Q5 Cloud Environment

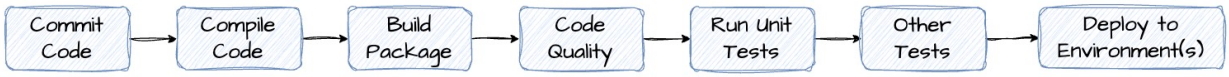
cloud environment

Q6 Multiple Environments



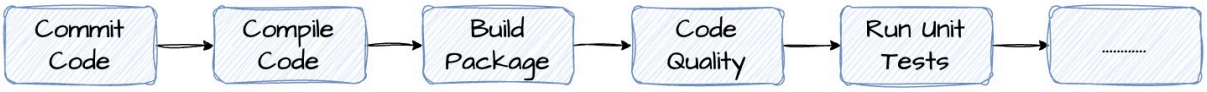
environments

Q7 Continuous Delivery/Deployment



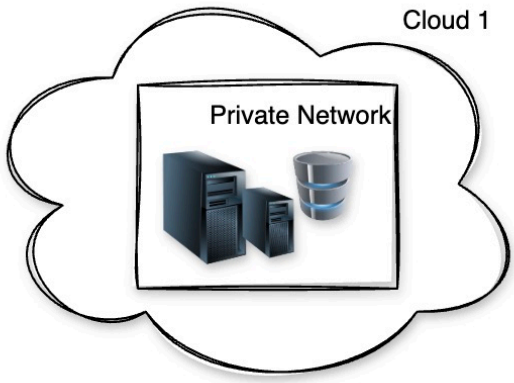
continuous deployment

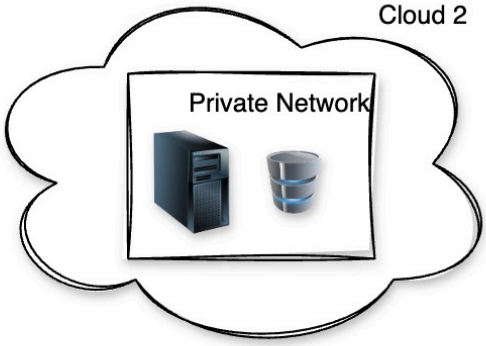
Q8 Continous Integration

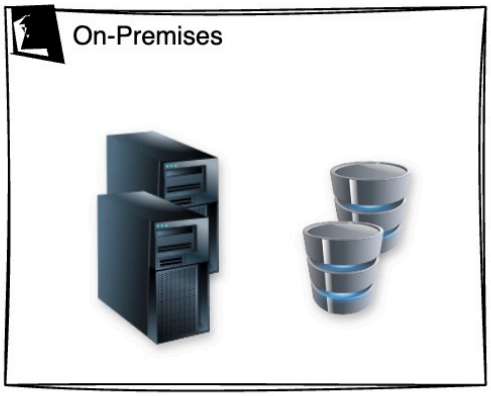


continuous integration

Q9 Hybrid Cloud







Q10 Container Orchestration

Container Orchestration